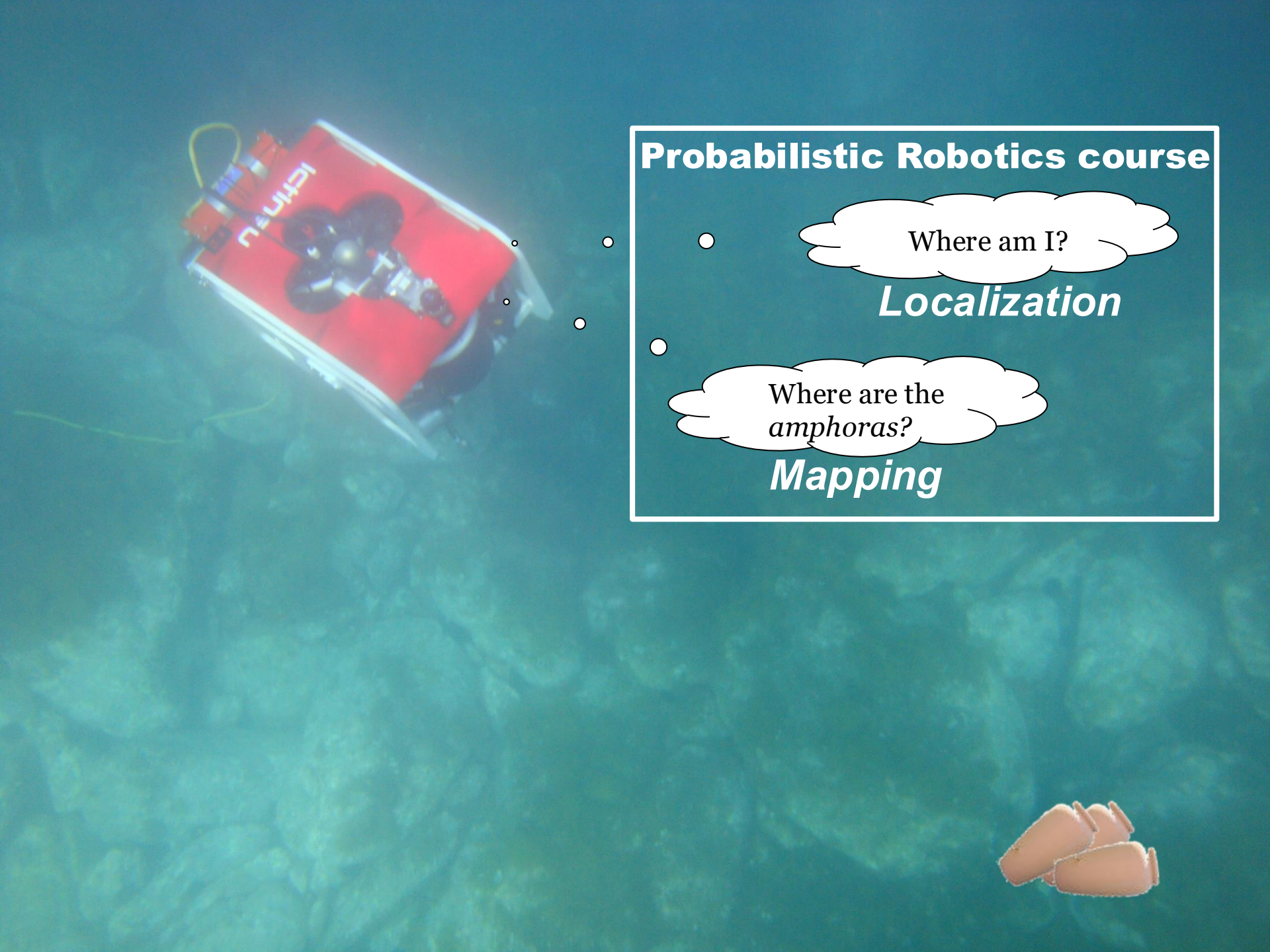
What is Probabilistic Robotics?

*'**Robotics** is the science of perceiving and manipulating the physical world through computer controlled mechanical devices.'*

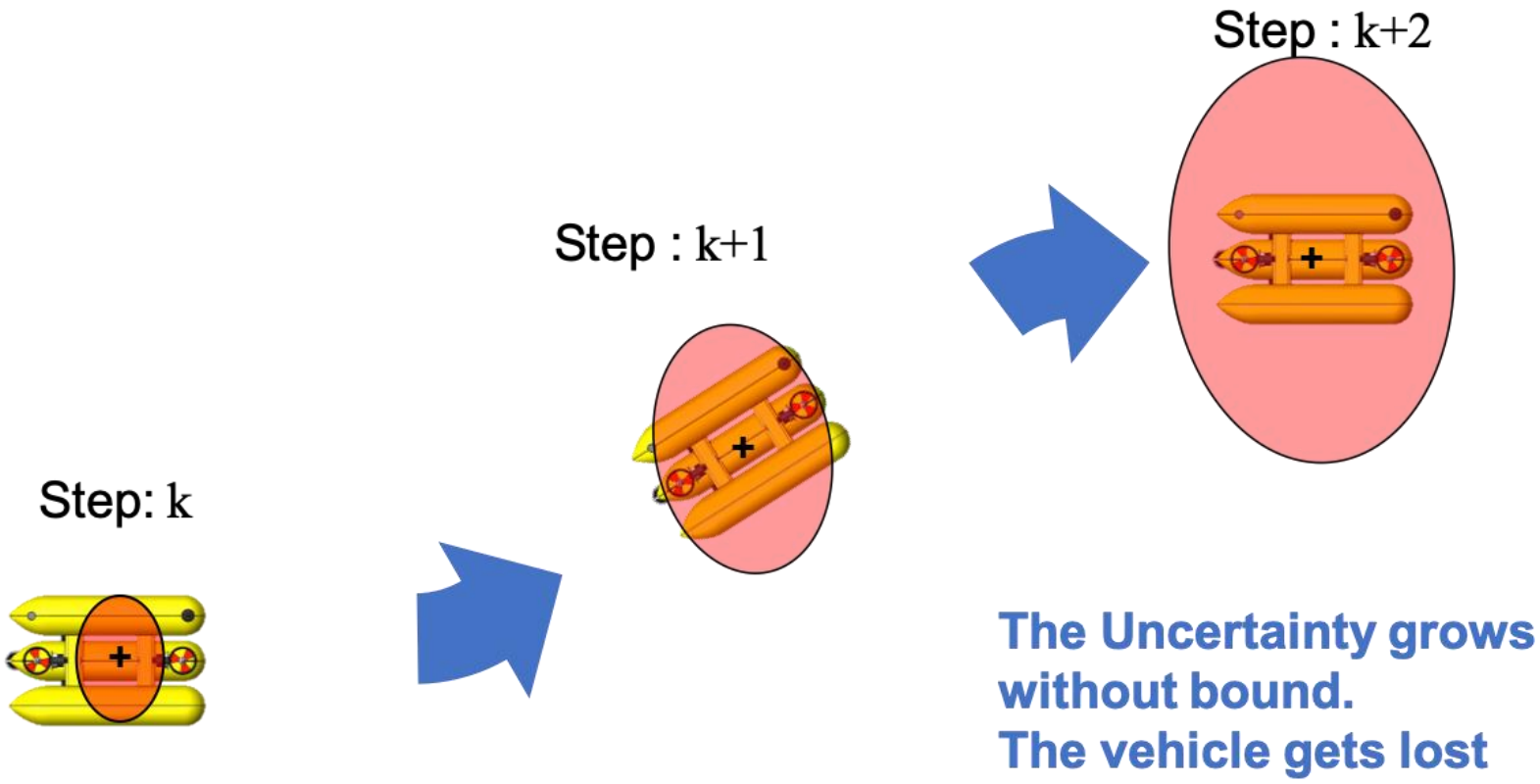
*'**Probabilistic robotics** is a relatively new approach to robotics that pays tribute to the uncertainty in robot perception and action.'*

Sebastian Thrun


Probabilistic Robotics course
Where am I?
Localization
Where are the
amphoras?
Mapping

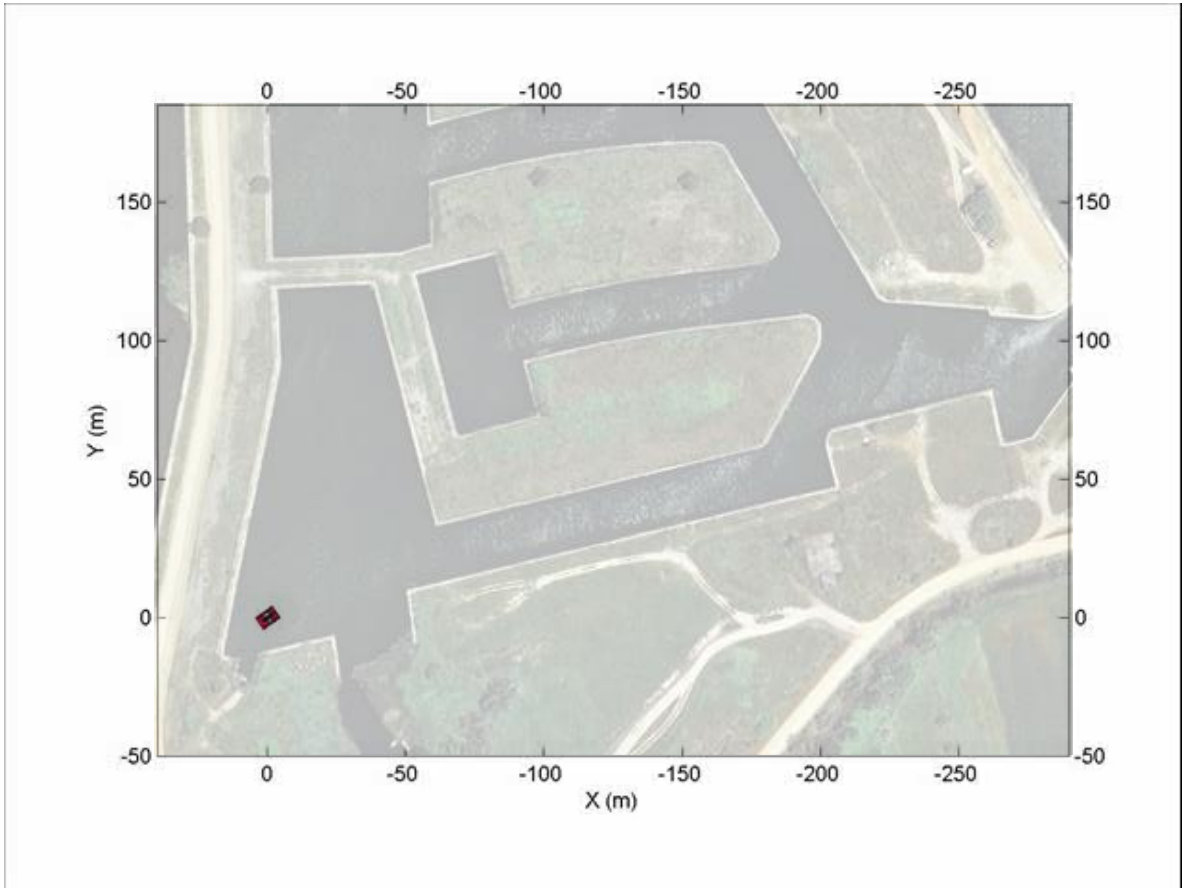
Introduction

- **Localization:** Estimate the position, orientation and velocity of a vehicle
- Localization through **Dead Reckoning**:



Introduction

- Localization through **Dead-Reckoning:**

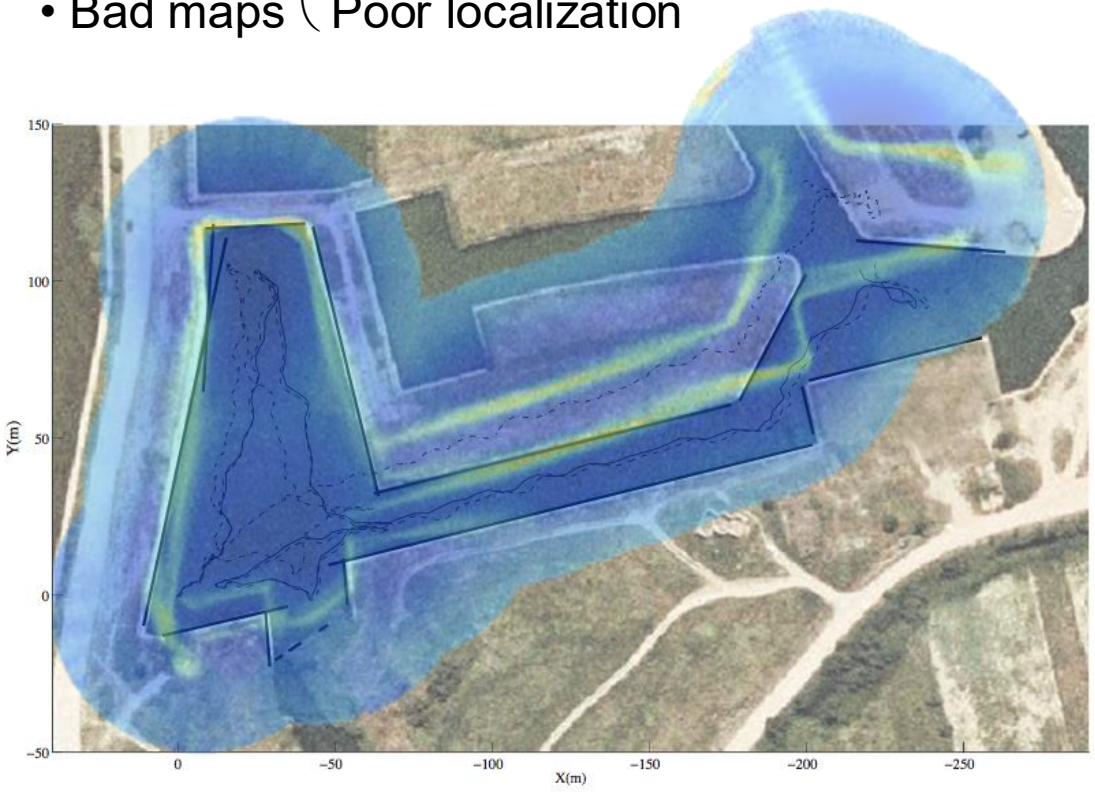


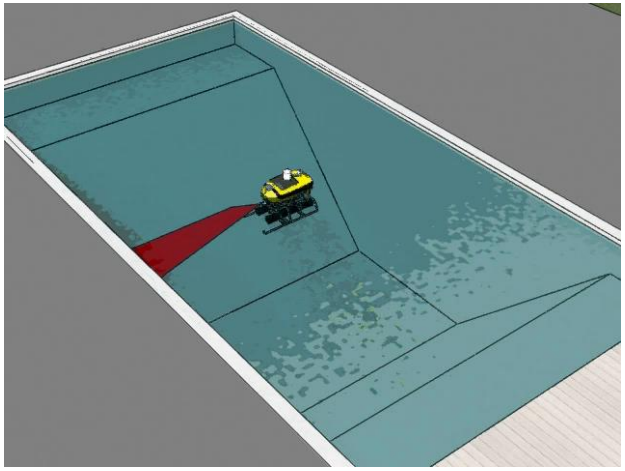


The robot position drifts

The Mapping Problem

- A conventional method for map building is incremental mapping.
  - Position Reference given by Dead Reckoning ➔ Map distorsion
- Bad maps ⟍ Poor localization



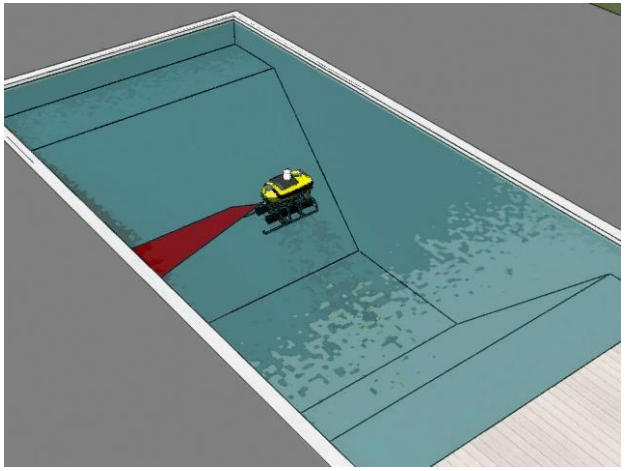


[Ribas 08]

The Mapping Problem

- A conventional method for map building is incremental mapping.
  - Position Reference given by Dead Reckoning ➔ Map distorsion
- Bad maps ⟍ Poor localization





[Ribas 08]

This is an old problem…

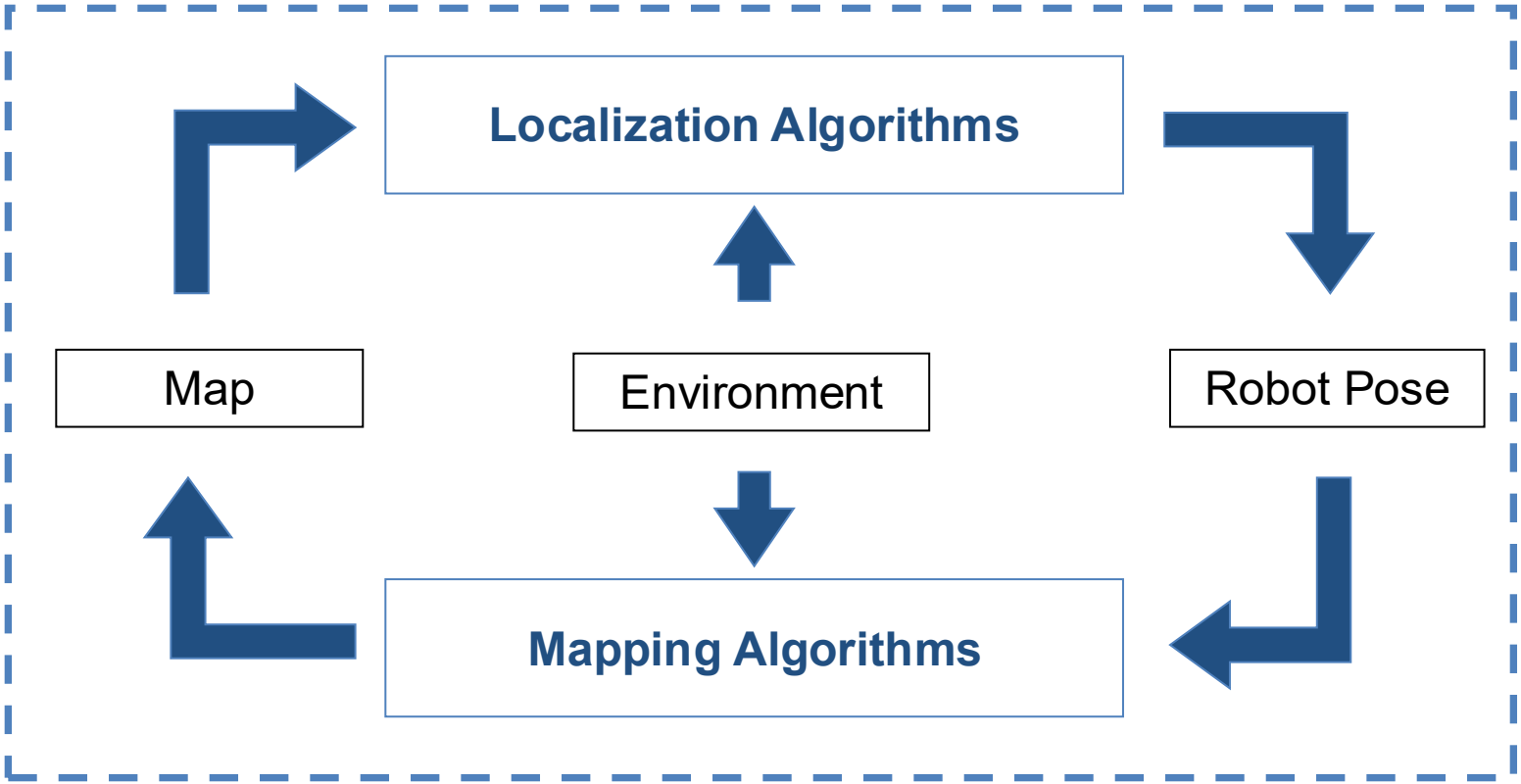


Ancient Europe. The **Catalan Atlas** of 1376.

This is an old problem…



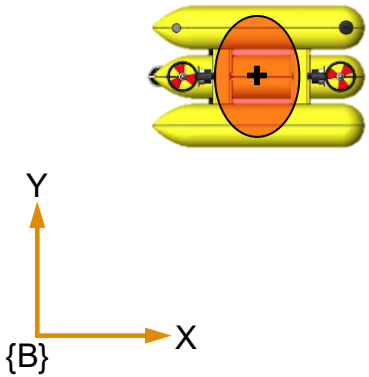
SLAM: Simultaneous Localization And Mapping

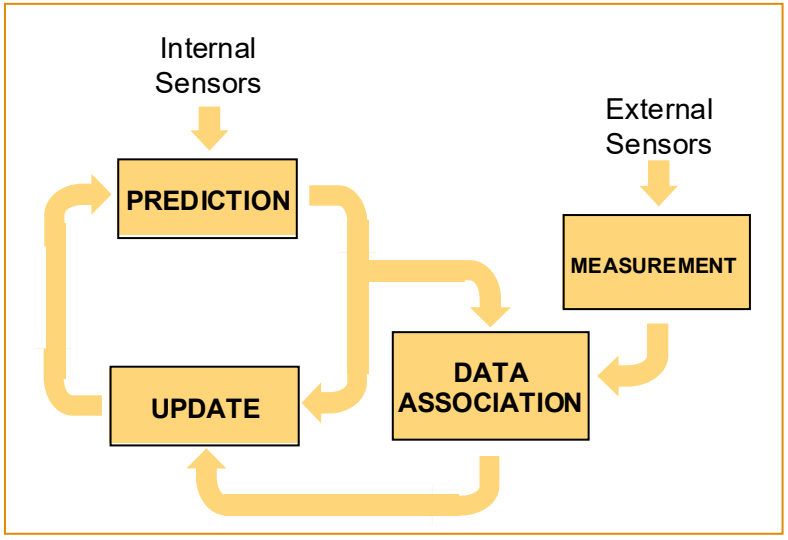


Introduction

> Localization Through **SLAM**:

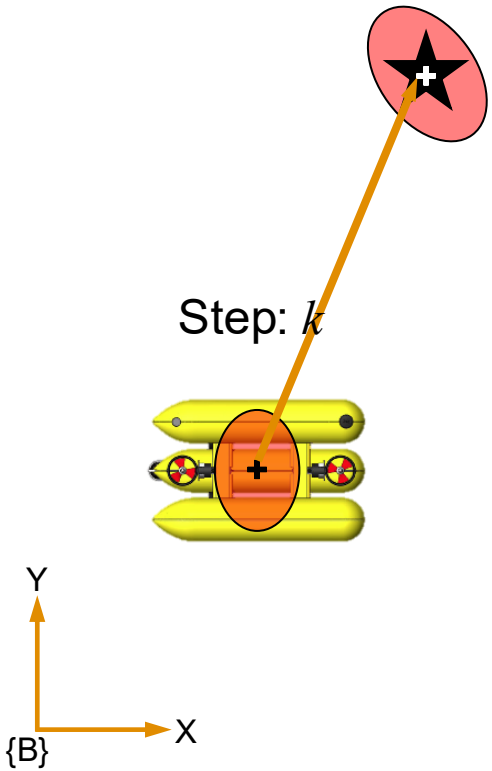
Step: k

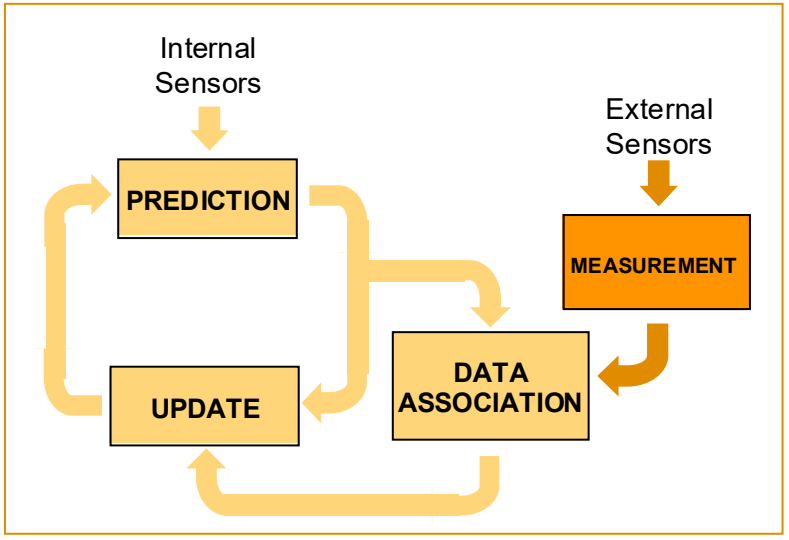




Introduction

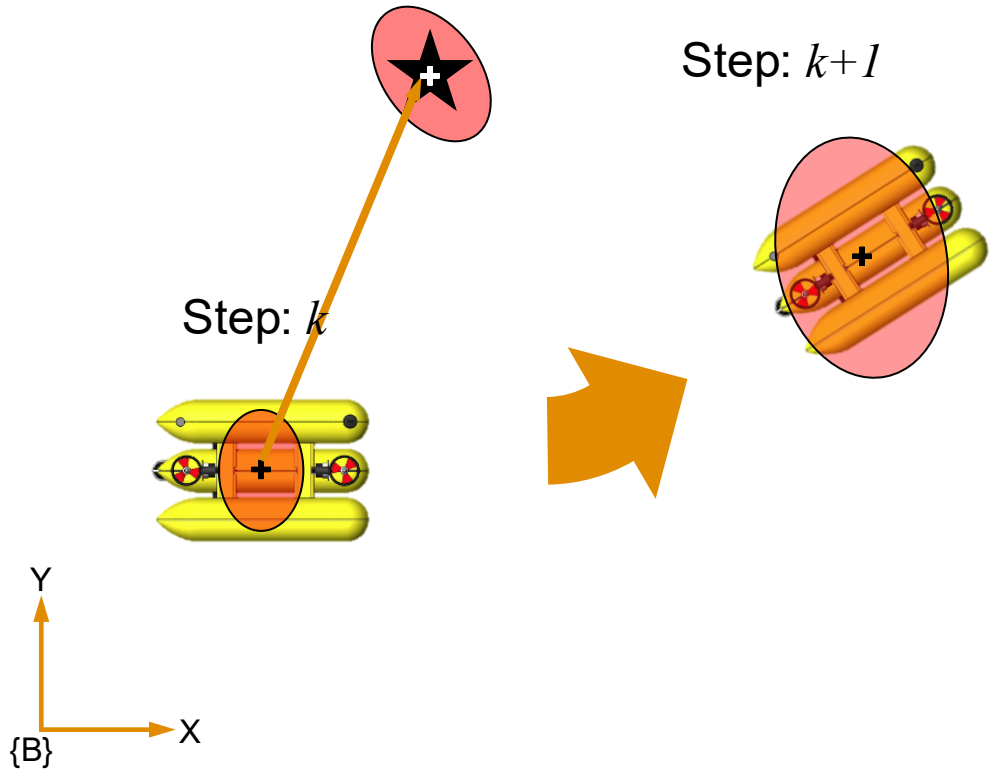
> Localization Through **SLAM**:

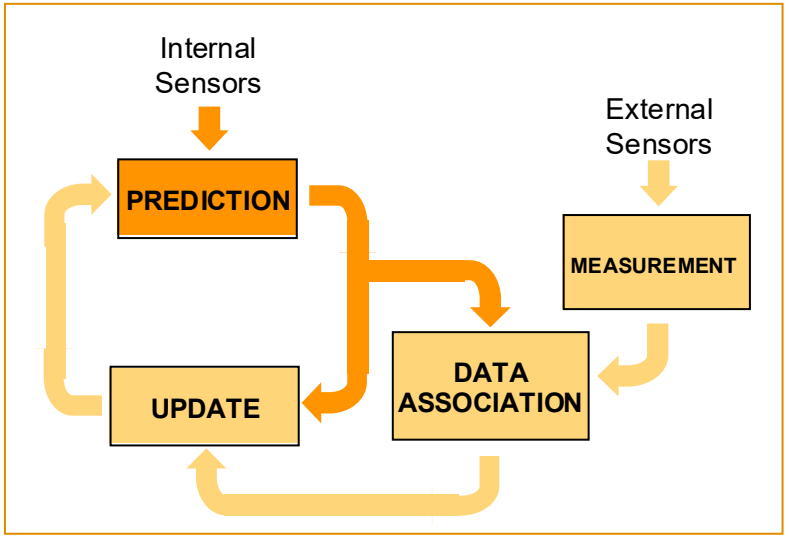




Introduction

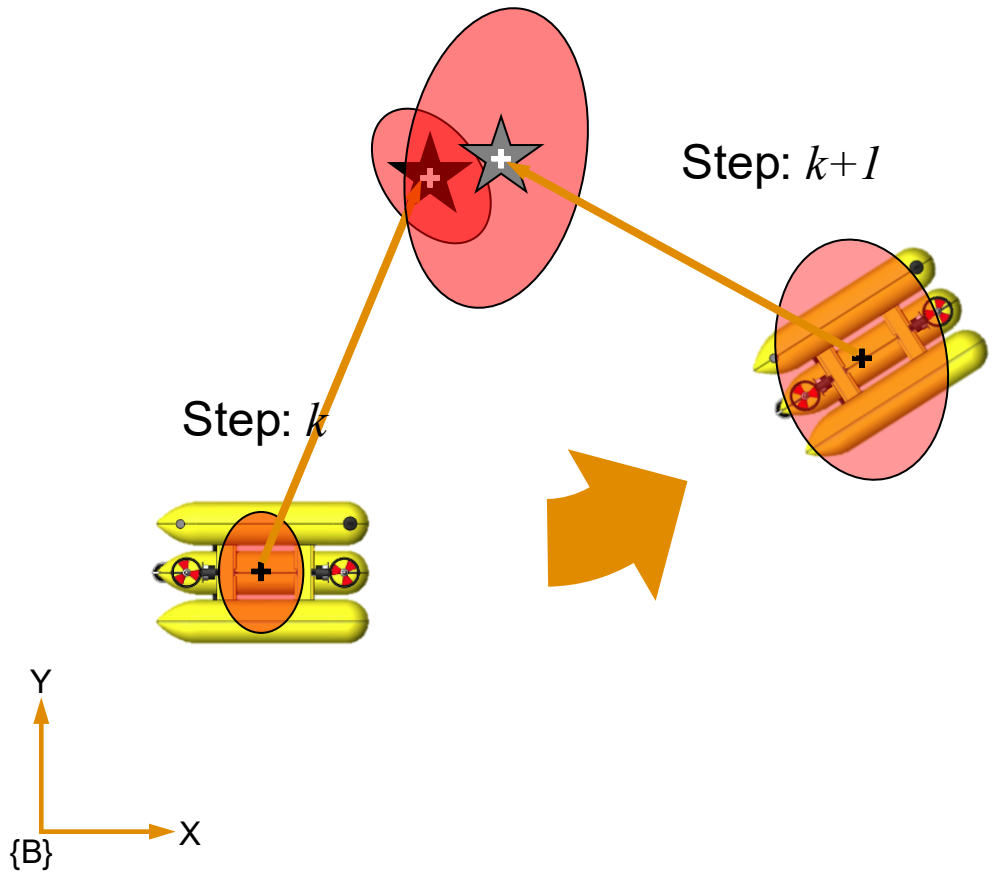
> Localization Through **SLAM**:

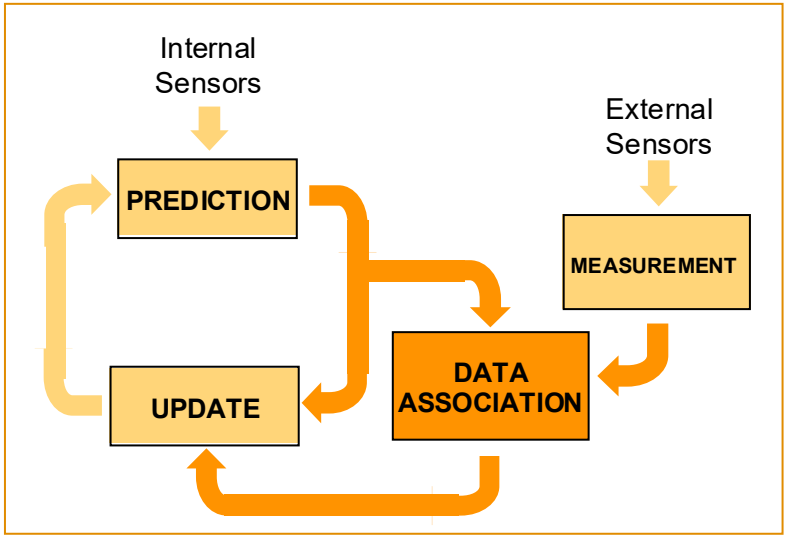




Introduction

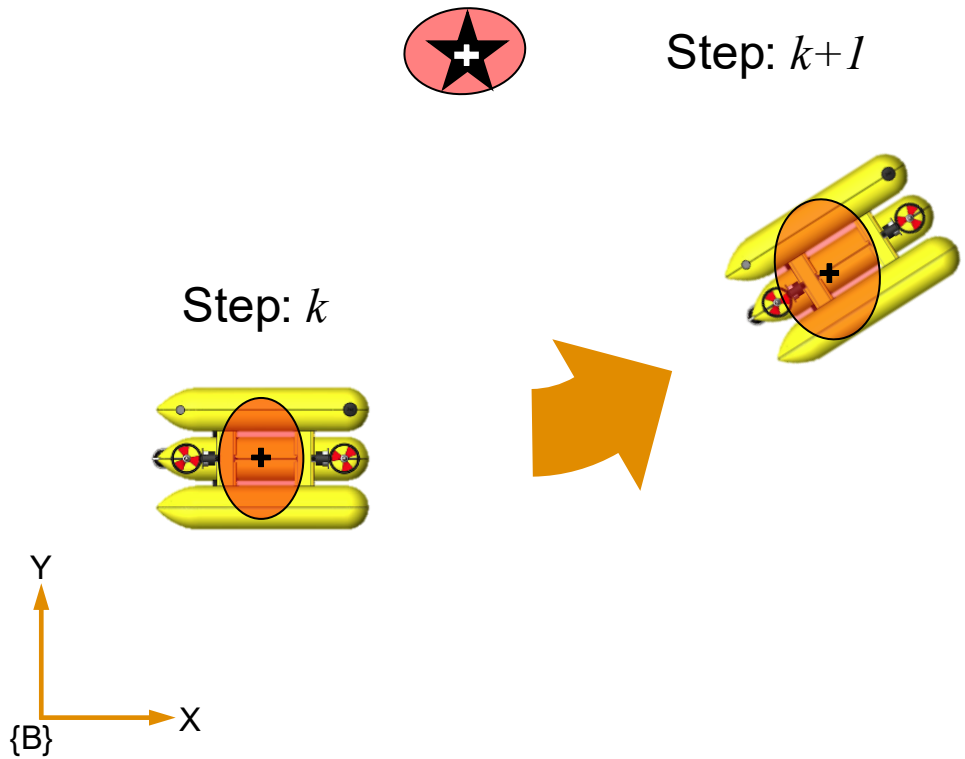
> Localization Through **SLAM**:

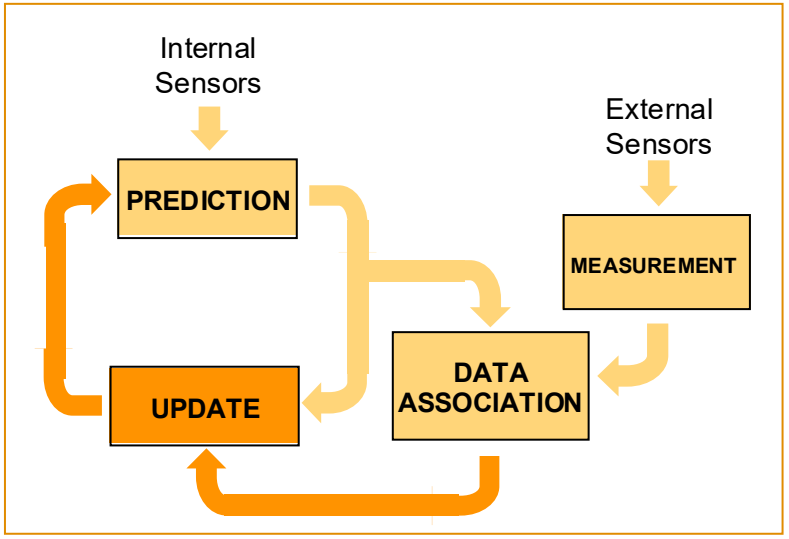




Introduction

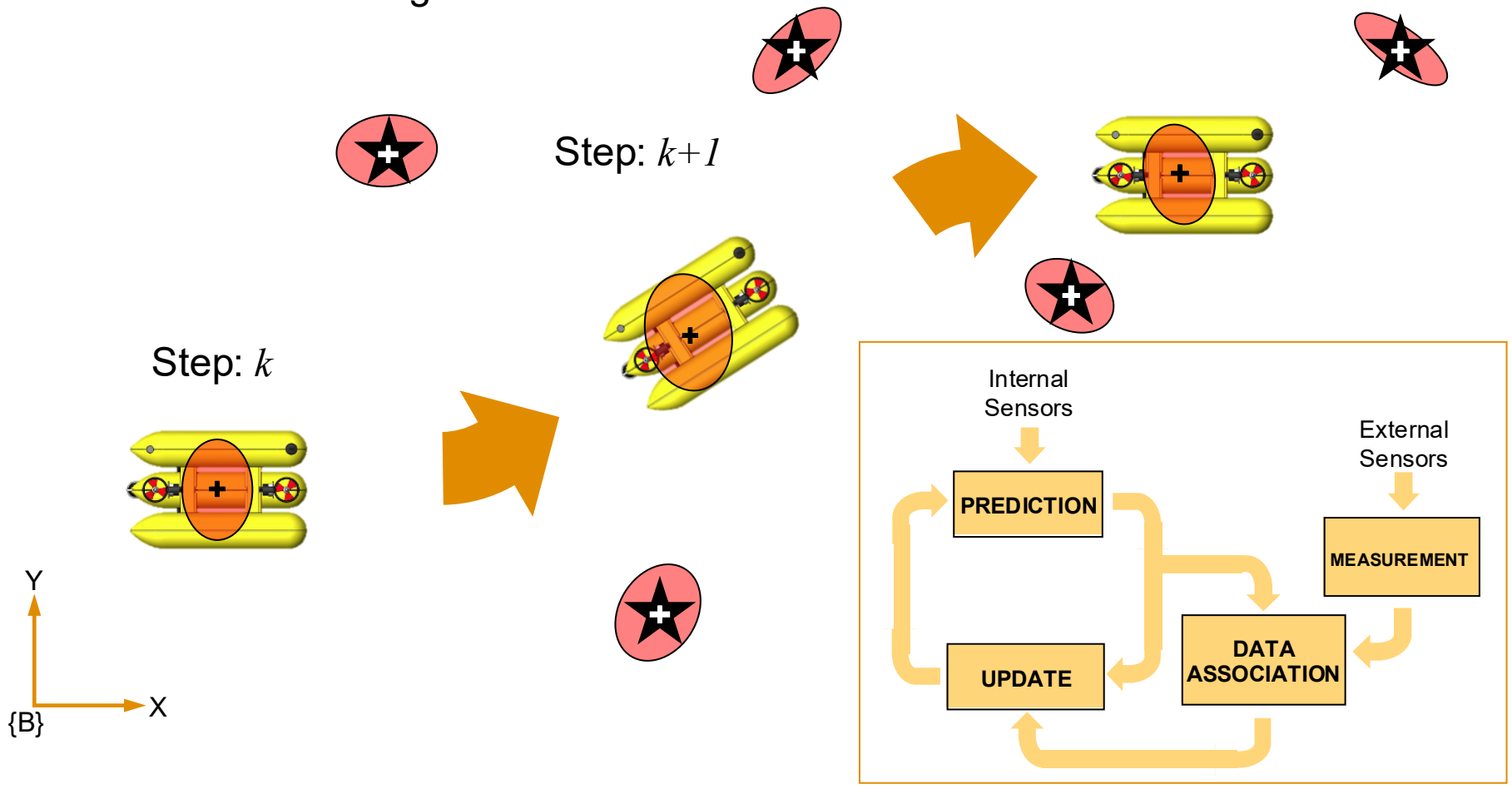
> Localization Through **SLAM**:





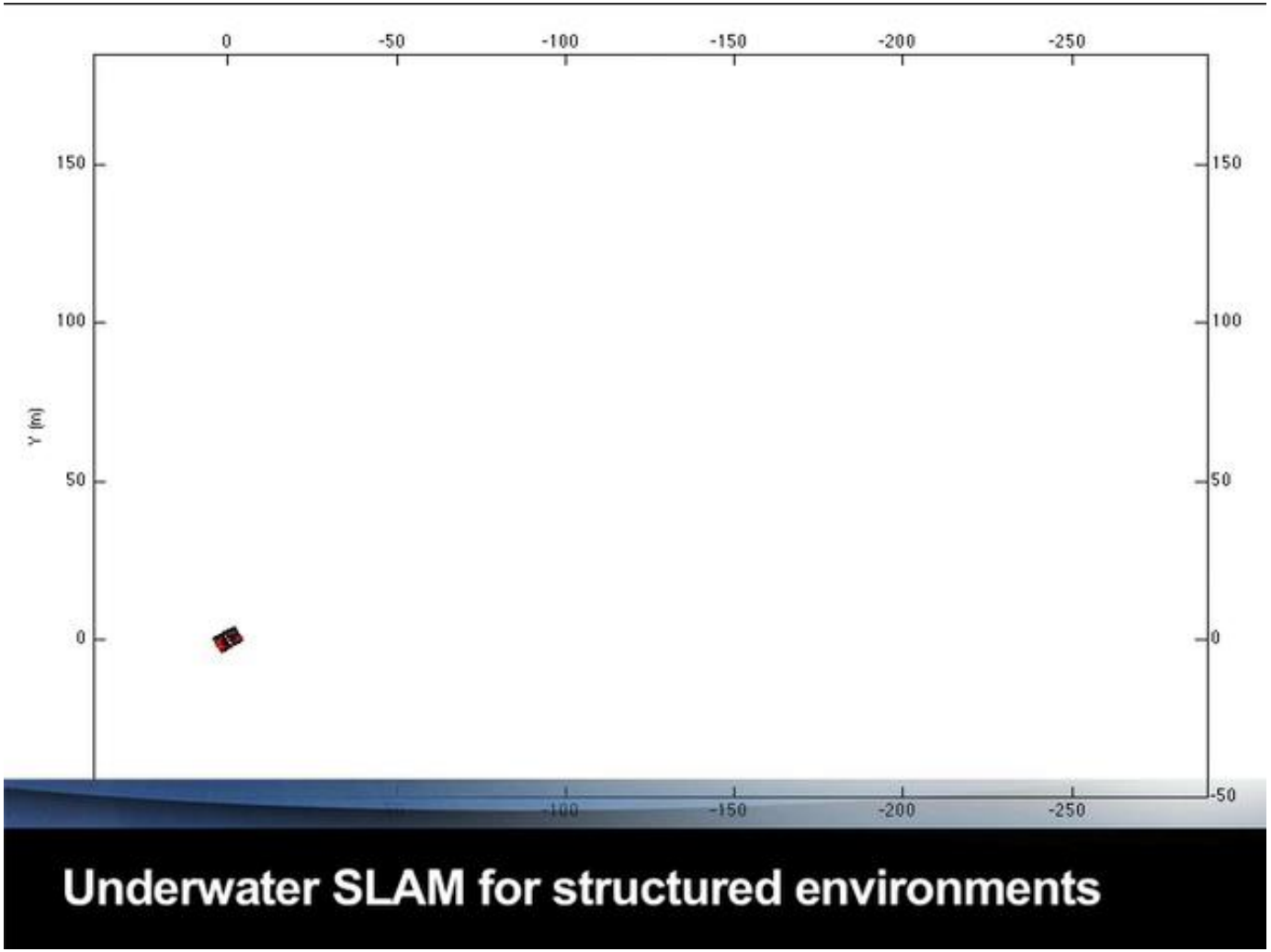
Introduction

> Localization Through **SLAM**:



Introduction

> Localization Through **SLAM**:



[Ribas 08]

Inspection of an Underwater Structure using Point Cloud SLAM with an AUV and a Laser Scanner

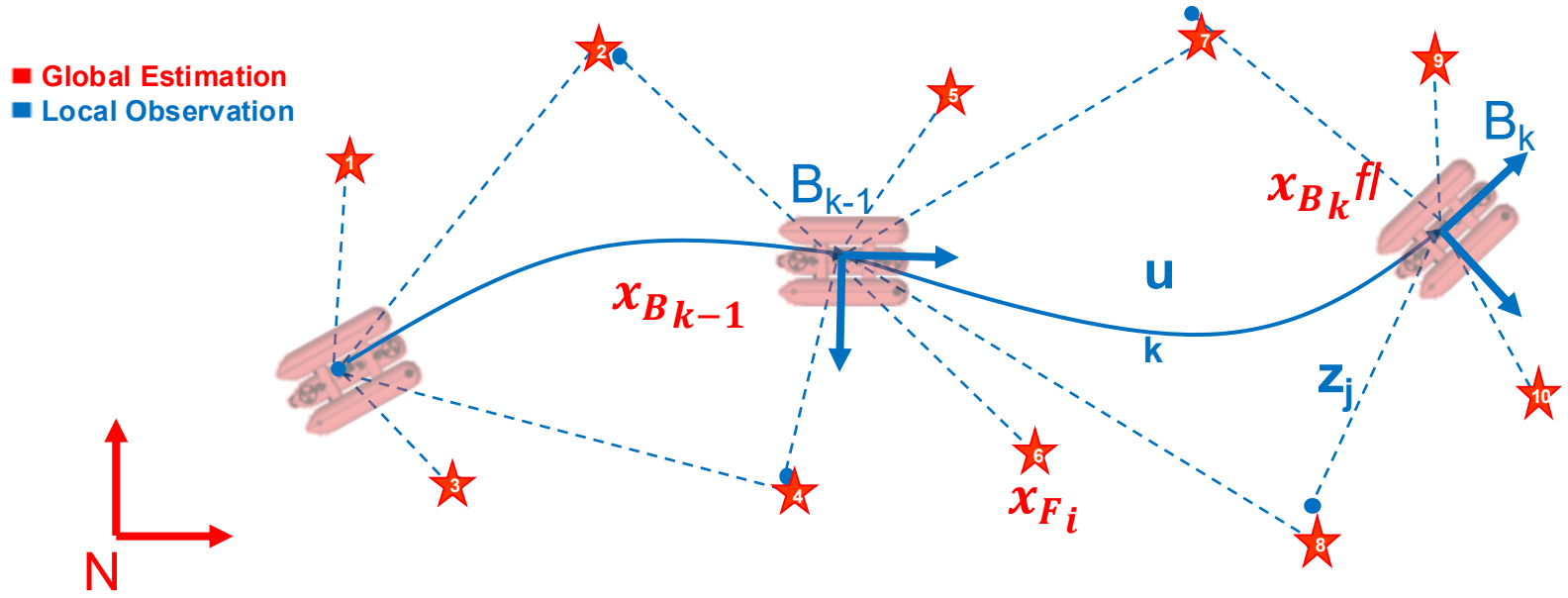
Albert Palomer, Pere Ridao and David Ribas

Universitat de Girona

[Palomer19]

Given a Subset of the Following Variables

- **Robot state** $\mathbf{x}_{0:k}$: trajectory composed of the robot pose and, optionally, its velocity.
- **Environment map m**: representation based on landmarks, occupancy grids, or geometric features.
- **Motion information** $\mathbf{u}_{1:k}$: control commands or proprioceptive measurements describing the robot motion.
- **Sensor observations** $\mathbf{z}_{1:k}$:
  - **Proprioceptive**: velocity, displacement, or pose measurements.
  - **Exteroceptive**: observations of landmarks or environmental features.



Solve for one of the following Problems

- **Dead reckoning**: Estimate the robot trajectory by integrating motion or proprioceptive measuremen

$$p(\mathbf{x}_{0:k} \mid \mathbf{u}_{1:k})$$

- **Localization**: Estimate the robot trajectory in a known map.

$$p(\mathbf{x}_{0:k} \mid \mathbf{z}_{1:k}, \mathbf{u}_{1:k}, \mathbf{m})$$

- **Mapping**: Estimate the map when the robot trajectory is known.

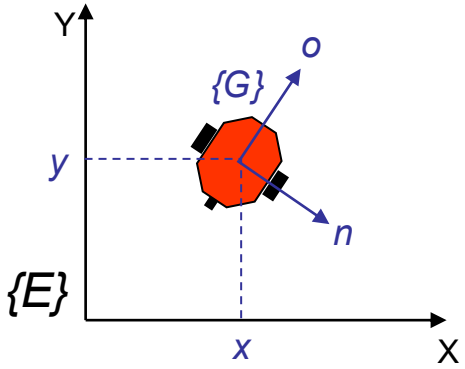
$$p(\mathbf{m} \mid \mathbf{z}_{1:k}, \mathbf{u}_{1:k}, \mathbf{x}_{0:k})$$

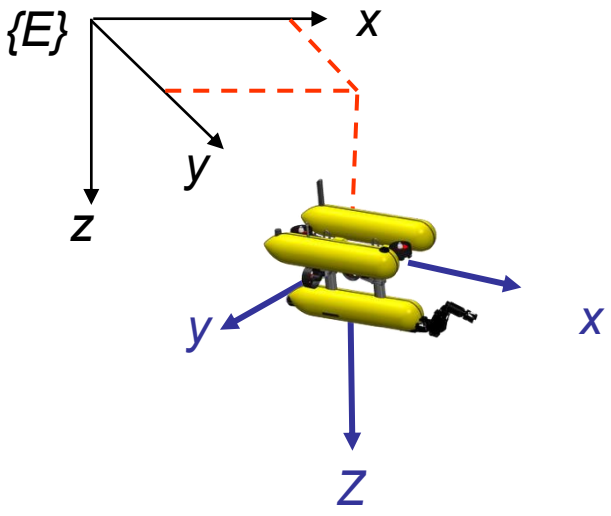
- **Simultaneous Localization and Mapping (SLAM)**: Jointly estimate the robot trajectory and the map.

$$p(\mathbf{x}_{0:k}, \mathbf{m} \mid \mathbf{z}_{1:k}, \mathbf{u}_{1:k})$$

Problem Statement

- **Pose:**
  - 2D: ${}^{E}(x,y,\theta)^{T}$
  - 3D: ${}^{E}(x, y, z,\phi,\theta,\varphi)^{T}$





- **Environment**
  - **Static:** only robot pose changes
  - **Dynamic:** the robot as well as the pose of other entities change

- **Localization:**
  - **Pasive:** Localization module only observes
  - **Active:** Robot is guided in a way that minimizes the localization error.

Introduction

What we want...

Algorithm 1: Localization Class

```
1  Class Localization
2      Pure Virtual GetInput() → u_k
3      Pure Virtual f(x_{k-1}, u_k) → x_k
4      Method LocalizationLoop(x_0)
5          x_{k-1} = x_0;
6          for k = 1 to ∞ do
7              x_k = Localize(x_{k-1});
8      Method Localize(x_{k-1})
9          u_k = GetInput();
10         x_k = f(x_{k-1}, u_k);
11         return x_k;
```

Sensor input to the Localization

Motion Model

Localization Loop

Localization Iteration

Introduction

Dead Reckoning



3 DOF Mobile Robot



4 DOF AUV

Dead Reckoning

Reference Frames, Pose & Velocity

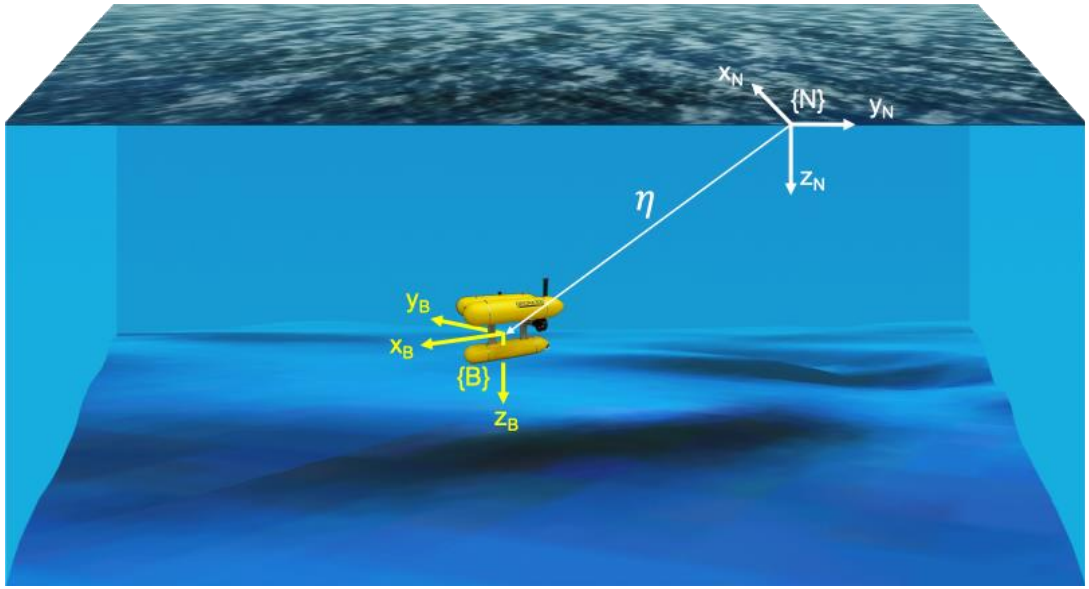
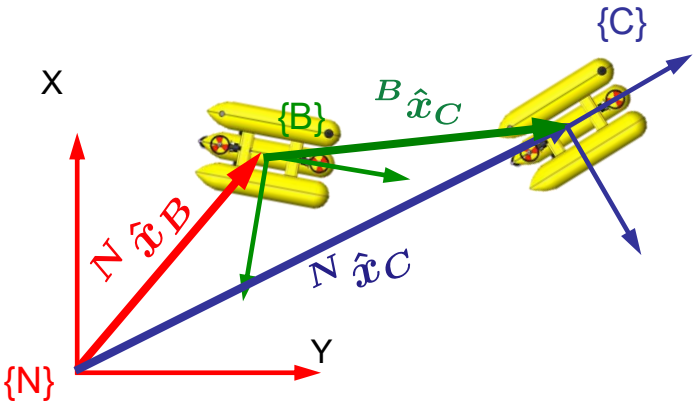


Fig. 1.2. Reference frames: NED (*N − f rame*) and body-fixed (*B − f rame*))

- N-Frame: NED Earth fixed frame
- B-Frame: AUV body-fixed frame
- ${}^N\eta_B = [{}^N x_B \ {}^N y_B \ {}^N z_B \ \psi_B]^T \in \mathbb{R}^4$: vehicle pose in the N-frame.
- ${}^B\nu = [{}^B u \ {}^B v \ {}^B w \ {}^B r]^T \in \mathbb{R}^4$: velocitiy in the B-frame.
- ${}^N\dot{\eta}_B = [{}^N\dot{x}_B \ {}^N\dot{y}_B \ {}^N\dot{z}_B \ {}^N\dot{\psi}_B]^T \in \mathbb{R}^4$: vehicle pose derivative in N-frame.

Using Rotation Matrices



$$B \equiv B \ at \ k-1$$
$$C \equiv B \ at \ k$$

Pose Compounding

> Given 2 poses

$$^{N}\boldsymbol{x}_{B} = [^{N}x_{B} \quad ^{N}y_{B} \quad ^{N}z_{B} \quad ^{N}\psi_{B}]^{T}$$
$$^{B}\boldsymbol{x}_{C} = [^{B}x_{C} \quad ^{B}y_{C} \quad ^{B}z_{C} \quad ^{B}\psi_{C}]^{T}$$

> The compounded pose is given by:

$$^{N}\boldsymbol{x}_{C} = {}^{N}\boldsymbol{x}_{B} \oplus {}^{B}\boldsymbol{x}_{C}$$

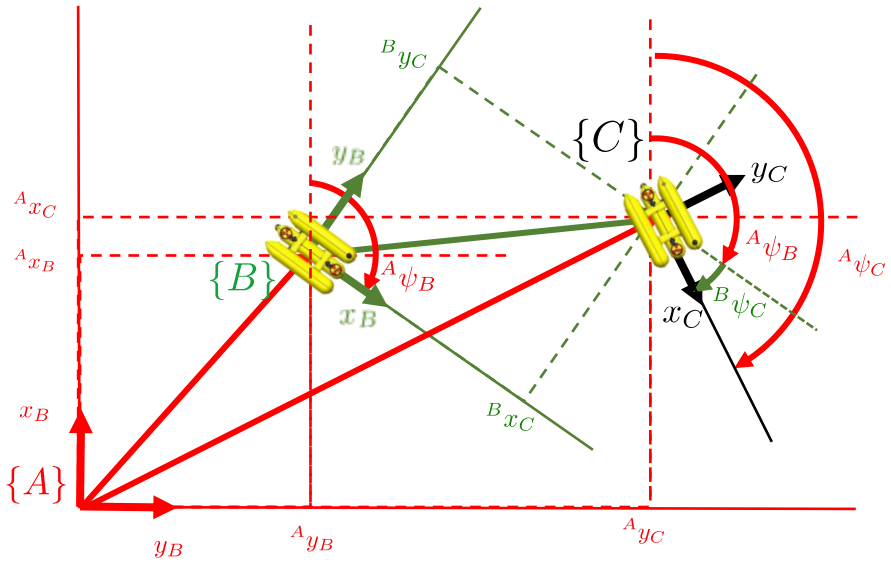
$$= \begin{bmatrix} ^{N}x_{B} \\ ^{N}y_{B} \\ ^{N}z_{B} \\ ^{N}\psi_{B} \end{bmatrix} \oplus \begin{bmatrix} ^{B}x_{C} \\ ^{B}y_{C} \\ ^{B}z_{C} \\ ^{B}\psi_{C} \end{bmatrix}$$

$$= \begin{bmatrix} \begin{bmatrix} ^{N}x_{B} \\ ^{N}y_{B} \\ ^{N}z_{B} \end{bmatrix} + \begin{bmatrix} \cos{}^{N}\psi_{B} & -\sin{}^{N}\psi_{B} & 0 \\ \sin{}^{N}\psi_{B} & \cos{}^{N}\psi_{B} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} ^{B}x_{C} \\ ^{B}y_{C} \\ ^{B}z_{C} \end{bmatrix} \\ {}^{N}\psi_{B} + {}^{B}\psi_{C} \end{bmatrix}$$

$$= \begin{bmatrix} ^{N}x_{B} + {}^{B}x_{C}\cos{}^{N}\psi_{B} - {}^{B}y_{C}\sin{}^{N}\psi_{B} \\ ^{N}y_{B} + {}^{B}x_{C}\sin{}^{N}\psi_{B} + {}^{B}y_{C}\cos{}^{N}\psi_{B} \\ ^{N}z_{B} + {}^{B}z_{C} \\ ^{N}\psi_{B} + {}^{B}\psi_{C} \end{bmatrix}$$

Dead Reckoning

From Geometry



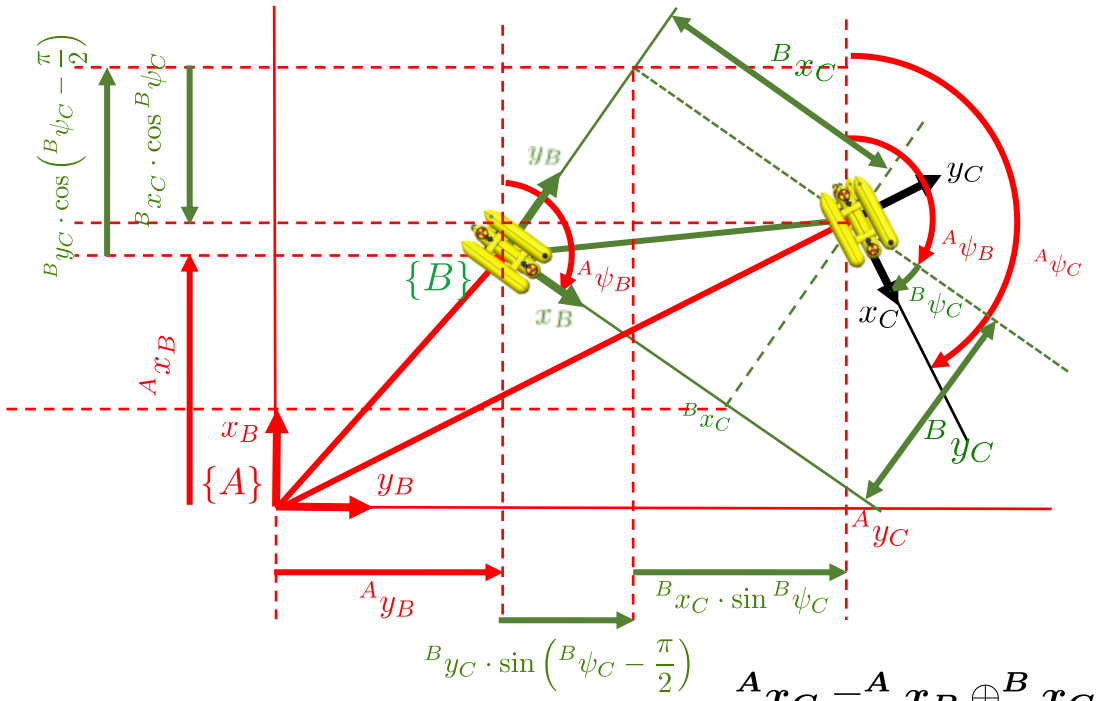
> Given 2 poses

- $\boldsymbol{{}^{A}x_B} = [{}^{A}x_B \;\; {}^{A}y_B \;\; {}^{A}z_B \;\; {}^{A}\psi_B]^T$
- $\boldsymbol{{}^{B}x_C} = [{}^{B}x_C \;\; {}^{B}y_C \;\; {}^{B}z_C \;\; {}^{B}\psi_C]^T$

$$\boldsymbol{{}^{A}x_C} = {}^{\boldsymbol{A}}\boldsymbol{x_B} \oplus {}^{\boldsymbol{B}}\boldsymbol{x_C} = [{}^{A}x_C \;\; {}^{A}y_C \;\; {}^{A}z_C \;\; {}^{A}\psi_C]^T$$

Dead Reckoning

From Geometry



$$ {}^{A}\boldsymbol{x_C} = {}^{A}\boldsymbol{x_B} \oplus {}^{B}\boldsymbol{x_C} $$

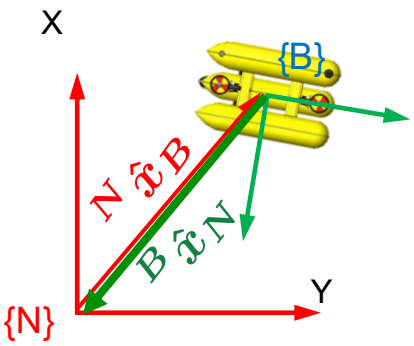
> Given 2 poses

- $ {}^{A}\boldsymbol{x_B} = [{}^{A}x_B \quad {}^{A}y_B \quad {}^{A}z_B \quad {}^{A}\psi_B]^T $
- $ {}^{B}\boldsymbol{x_C} = [{}^{B}x_C \quad {}^{B}y_C \quad {}^{B}z_C \quad {}^{B}\psi_C]^T $

$$ = \begin{bmatrix} {}^{A}x_B \\ {}^{A}y_B \\ {}^{A}z_B \\ {}^{A}\psi_B \end{bmatrix} \oplus \begin{bmatrix} {}^{B}x_C \\ {}^{B}y_C \\ {}^{B}z_C \\ {}^{B}\psi_C \end{bmatrix} $$

$$ = \begin{bmatrix} {}^{A}x_B + {}^{B}x_C \cos {}^{A}\psi_B - {}^{B}y_C \sin {}^{A}\psi_B \\ {}^{A}y_B + {}^{B}x_C \sin {}^{A}\psi_B + {}^{B}y_C \cos {}^{A}\psi_B \\ {}^{A}z_B + {}^{B}z_C \\ {}^{A}\psi_B + {}^{B}\psi_C \end{bmatrix} $$

Using Rotation Matrices



Pose Inversion

> Given a transformation

$$ {}^{N}\boldsymbol{x}_{B} = [{}^{N}x_B \quad {}^{N}y_B \quad {}^{N}z_B \quad {}^{N}\psi_B]^T $$

> The inverse transformation is given by:

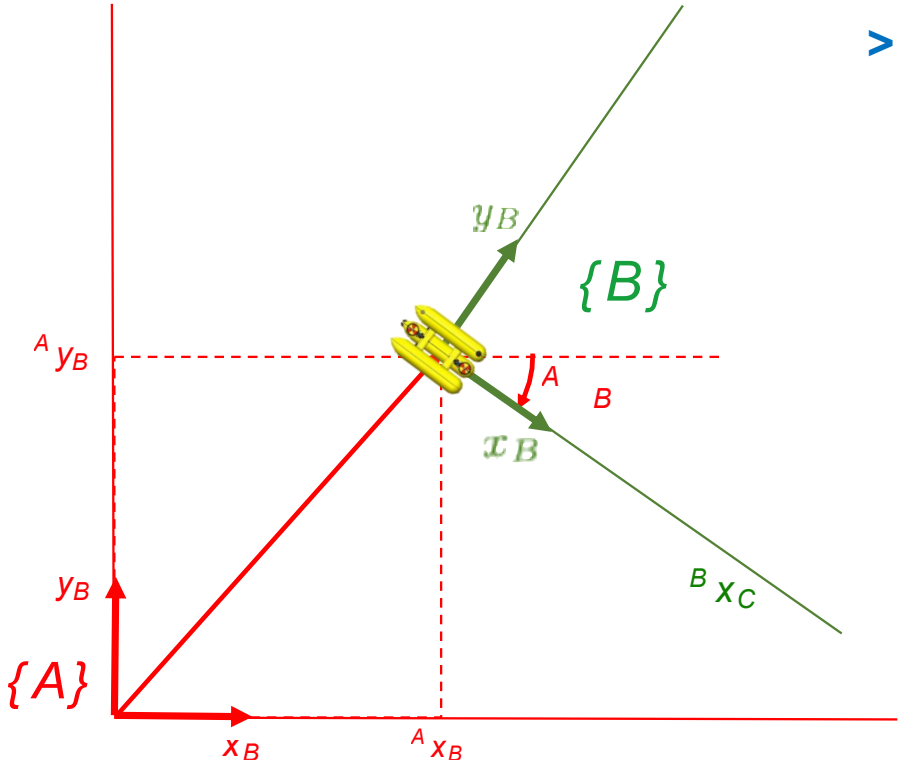
$$ {}^{B}\boldsymbol{x}_{N} = \ominus {}^{N}\boldsymbol{x}_{B} = \begin{bmatrix} -\begin{bmatrix} \cos({}^{N}\psi_B) & -\sin({}^{N}\psi_B) & 0 \\ \sin({}^{N}\psi_B) & \cos({}^{N}\psi_B) & 0 \\ 0 & 0 & 1 \end{bmatrix}^{-1} \cdot \begin{bmatrix} {}^{N}x_B \\ {}^{N}y_B \\ {}^{N}z_B \end{bmatrix} \\ -{}^{N}\psi_B \end{bmatrix} $$

$$ = \begin{bmatrix} \begin{bmatrix} \cos({}^{N}\psi_B) & \sin({}^{N}\psi_B) & 0 \\ -\sin({}^{N}\psi_B) & \cos({}^{N}\psi_B) & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} {}^{N}x_B \\ {}^{N}y_B \\ {}^{N}z_B \end{bmatrix} \\ -{}^{N}\psi_B \end{bmatrix} $$

$$ = \begin{bmatrix} -{}^{N}x_B \cos {}^{N}\psi_B - {}^{N}y_B \sin {}^{N}\psi_B \\ {}^{N}x_B \sin {}^{N}\psi_B - {}^{N}y_B \cos {}^{N}\psi_B \\ -{}^{N}z_B \\ -{}^{N}\psi_B \end{bmatrix} $$

Dead Reckoning

From Geometry

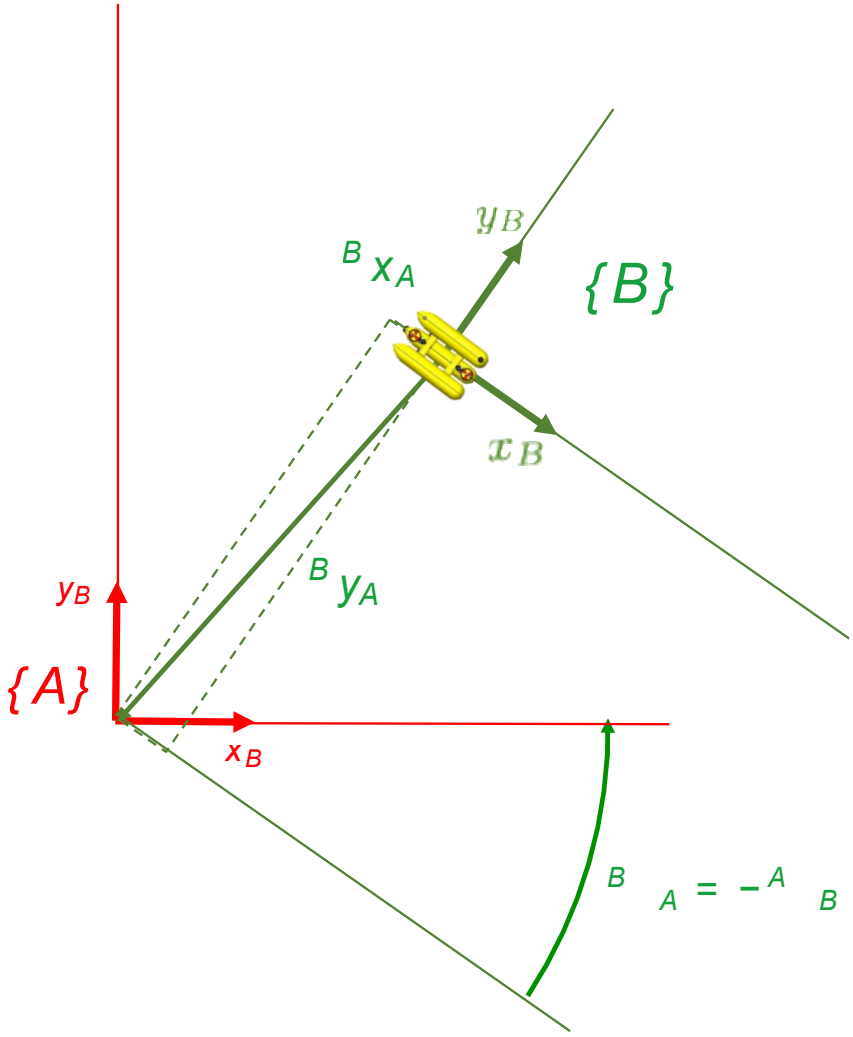


> Given 1 pose

- $^{A}\boldsymbol{x_B} = [^{A}x_B \;\; ^{A}y_B \;\; ^{A}z_B \;\; ^{A}\psi_B]^T$

Dead Reckoning

From Geometry



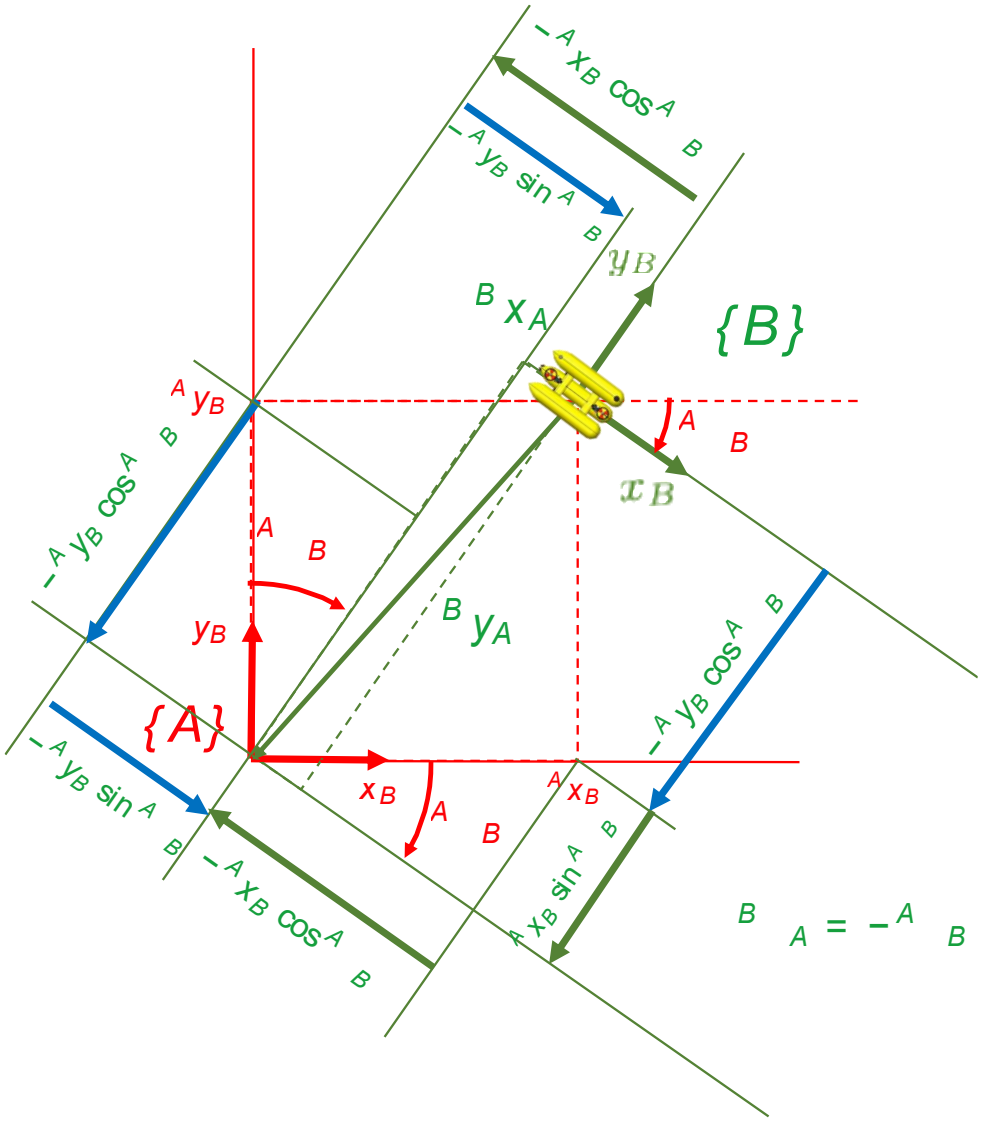
> Given 1 pose

- ${}^{\boldsymbol{A}}\boldsymbol{x_B} = [{}^A x_B \quad {}^A y_B \quad {}^A z_B \quad {}^A \psi_B]^T$

$${}^{\boldsymbol{B}}\boldsymbol{x_A} = \ominus {}^{\boldsymbol{A}}\boldsymbol{x_B} = [{}^B x_A \quad {}^B y_A \quad {}^B z_A \quad {}^B \psi_A]^T$$

Dead Reckoning

From Geometry



> Given 1 pose

- $^{A}\boldsymbol{x_B} = [^{A}x_B \quad ^{A}y_B \quad ^{A}z_B \quad ^{A}\psi_B]^T$

$$^{B}\boldsymbol{x_A} = \ominus\, ^{A}\boldsymbol{x_B} = [^{B}x_A \quad ^{B}y_A \quad ^{B}z_A \quad ^{B}\psi_A]^T$$

$$= \begin{bmatrix} -^{A}x_B \cos{}^{A}\psi_B - {}^{A}y_B \sin{}^{A}\psi_B \\ {}^{A}x_B \sin{}^{A}\psi_B - {}^{A}y_B \cos{}^{A}\psi_B \\ -^{A}z_B \\ -^{A}\psi_B \end{bmatrix}$$

Dead Reckoning

Pose Compounding Class

Algorithm 1: Pose4D Class

Class *Pose4D*

  Method `Pose4D(`AxB`)`

    $self.AxB = AxB$

  end

  Method `oplus(`BxC`)`

    $\begin{bmatrix} {}^{A}x_B & {}^{A}y_B & {}^{A}z_B & {}^{A}\psi_B \end{bmatrix}^T = self.AxB;$

    $\begin{bmatrix} {}^{B}x_C & {}^{B}_{y}C & Bz_C & {}^{B}\psi_C \end{bmatrix}^T = BxC;$

    $$AxC = \begin{bmatrix} {}^{A}x_B +^{B} x_C \cos {}^{A}\psi_B -^{B} y_C \sin {}^{A}\psi_B \\ {}^{A}y_B +^{B} x_C \sin {}^{A}\psi_B +^{B} y_C \cos {}^{A}\psi_B \\ {}^{A}z_B +^{B} z_C \\ {}^{A}\psi_B +^{B} \psi_C \end{bmatrix};$$

    return AxC

  end

  Method `ominus()`

    $\begin{bmatrix} {}^{A}x_B & {}^{A}y_B & {}^{A}z_B & {}^{A}\psi_B \end{bmatrix}^T = self.AxB;$

    $$^{B}x_A = \begin{bmatrix} -{}^{A}x_B \cos {}^{A}\psi_B -^{A} y_B \sin {}^{A}\psi_B \\ {}^{A}x_B \sin {}^{A}\psi_B -^{A} y_B \cos {}^{A}\psi_B \\ -{}^{A}z_B \\ -{}^{A}\psi_B \end{bmatrix};$$

    return BxA

  end

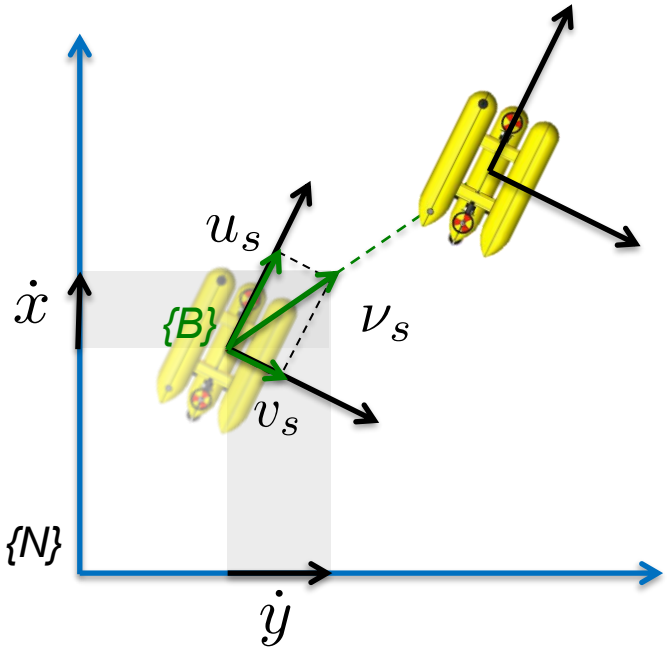
Use Example

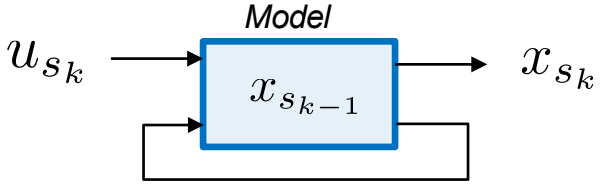
```
AxB = Pose4D([1;2;2;pi/2])
BxC = Pose4D([3;4;2;pi])

AxC = AxB.oplus(BxC)
```

Dead Reckoning

4 DOF AUV Simulation

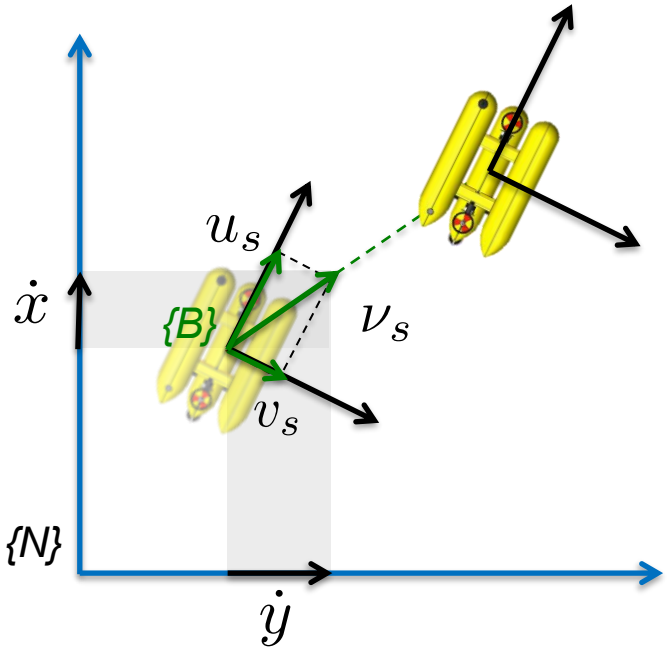




- N-Frame: NED Earth fixed frame
- B-Frame: AUV body-fixed frame
- ${}^N\eta_B = [{}^N x_B \ {}^N y_B \ {}^N z_B \ \psi_B]^T \in \mathbb{R}^4$: vehicle pose in the N-frame.
- ${}^B\nu = [{}^B u \ {}^B v \ {}^B w \ {}^B r]^T \in \mathbb{R}^4$: velocitiy in the B-frame.
- ${}^N\dot{\eta}_B = [{}^N\dot{x}_B \ {}^N\dot{y}_B \ {}^N\dot{z}_B \ {}^N\dot{\psi}_B]^T \in \mathbb{R}^4$: vehicle pose derivative in N-frame.

Dead Reckoning

4 DOF AUV Simulation



Model state

$$\eta_{s_{k-1}} = [x_{s_{k-1}} \quad y_{s_{k-1}} \quad z_{s_{k-1}} \quad \psi_{s_{k-1}}]^T$$
$$\nu_{s_{k-1}} = [u_{s_{k-1}} \quad v_{s_{k-1}} \quad w_{s_{k-1}} \quad r_{s_{k-1}}]^T$$
$$x_{s_{k-1}} = [\eta_{s_{k-1}}^T \quad \nu_{s_{k-1}}^T]^T$$

Model input

$$u_{s_k} = \nu_d = [u_d \quad v_d \quad w_d \quad r_d]^T$$

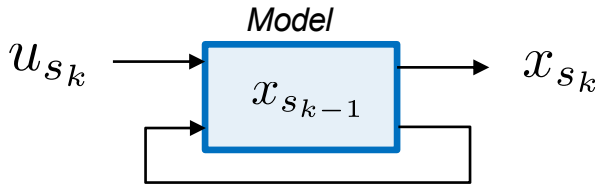
Model noise

$$w_{s_k} = \dot{\nu}_{s_k}$$

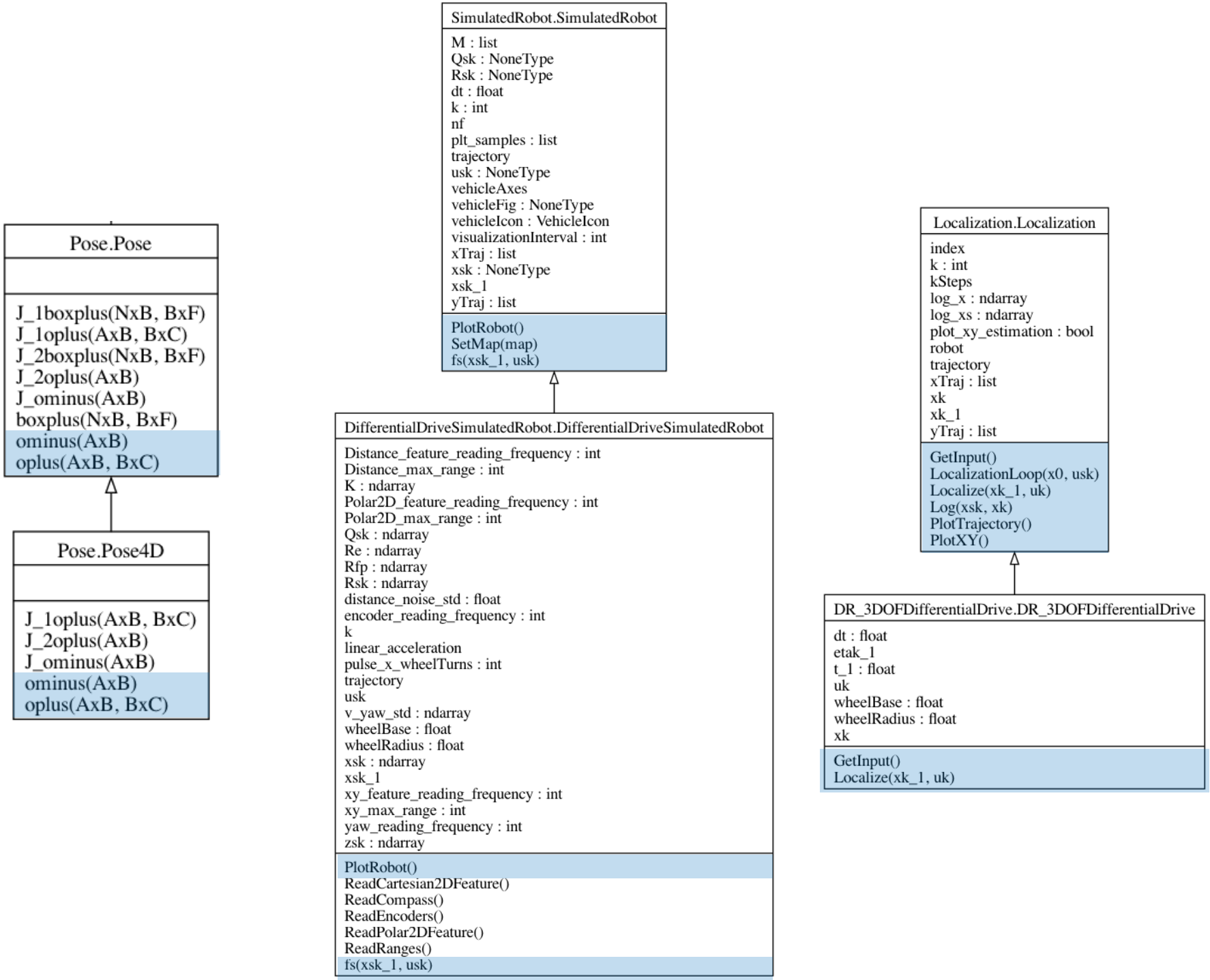
Motion model

$$x_{s_k} = f_s(x_{s_{k-1}}, u_{s_k}, w_{s_k}) = \begin{bmatrix} \eta_{s_{k-1}} \oplus (\nu_{s_{k-1}}\Delta t + \frac{1}{2} w_{s_k}\Delta t^2) \\ \nu_{s_{k-1}} + K(\nu_d - \nu_{s_{k-1}}) + w_{s_k}\Delta t \end{bmatrix};$$

$$K = diag(k_1, k_2, k_3, k_4) \quad k_i > 0$$



- N-Frame: NED Earth fixed frame
- B-Frame: AUV body-fixed frame
- ${}^N\eta_B = [{}^N x_B \ {}^N y_B \ {}^N z_B \ \psi_B]^T \in \mathbb{R}^4$: vehicle pose in the N-frame.
- ${}^B\nu = [{}^B u \ {}^B v \ {}^B w \ {}^B r]^T \in \mathbb{R}^4$: velocitiy in the B-frame.
- ${}^N\dot{\eta}_B = [{}^N\dot{x}_B \ {}^N\dot{y}_B \ {}^N\dot{z}_B \ {}^N\dot{\psi}_B]^T \in \mathbb{R}^4$: vehicle pose derivative in N-frame.


Pose.Pose
J_1boxplus(NxB, BxF)
J_1oplus(AxB, BxC)
J_2boxplus(NxB, BxF)
J_2oplus(AxB)
J_ominus(AxB)
boxplus(NxB, BxF)
ominus(AxB)
oplus(AxB, BxC)
Pose.Pose4D
J_1oplus(AxB, BxC)
J_2oplus(AxB)
J_ominus(AxB)
ominus(AxB)
oplus(AxB, BxC)
SimulatedRobot.SimulatedRobot
M : list
Qsk : NoneType
Rsk : NoneType
dt : float
k : int
nf
plt_samples : list
trajectory
usk : NoneType
vehicleAxes
vehicleFig : NoneType
vehicleIcon : VehicleIcon
visualizationInterval : int
xTraj : list
xsk : NoneType
xsk_1
yTraj : list
PlotRobot()
SetMap(map)
fs(xsk_1, usk)
DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot
Distance_feature_reading_frequency : int
Distance_max_range : int
K : ndarray
Polar2D_feature_reading_frequency : int
Polar2D_max_range : int
Qsk : ndarray
Re : ndarray
Rfp : ndarray
Rsk : ndarray
distance_noise_std : float
encoder_reading_frequency : int
k
linear_acceleration
pulse_x_wheelTurns : int
trajectory
usk
v_yaw_std : ndarray
wheelBase : float
wheelRadius : float
xsk : ndarray
xsk_1
xy_feature_reading_frequency : int
xy_max_range : int
yaw_reading_frequency : int
zsk : ndarray
PlotRobot()
ReadCartesian2DFeature()
ReadCompass()
ReadEncoders()
ReadPolar2DFeature()
ReadRanges()
fs(xsk_1, usk)
Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)

Dead Reckoning

4 DOF AUV Simulation

Algorithm 1: SimulatedRobot Class

```
1  Class SimulatedRobot
2      Method SimulatedRobot(xs0, map)
3          self.xs0 = xs0;                    // initial simulation state
4          self.M = map;                      // Feature Map
5      end
6      Pure Virtual fs(x_{k-1}, u_k) → x_k ;   // To be overridden
7      Method PlotRobot(x_{k-1}, u_k)
8          ...;      // Plots the robot at the current simulated pose
9      end
10     Method SetMap(x_{s_k}, x_k)
11         self.M = map;
12     end
```

Model state

$$\eta_{s_{k-1}} = \begin{bmatrix} x_{s_{k-1}} & y_{s_{k-1}} & z_{s_{k-1}} & \psi_{s_{k-1}} \end{bmatrix}^T$$

$$\nu_{s_{k-1}} = \begin{bmatrix} u_{s_{k-1}} & v_{s_{k-1}} & w_{s_{k-1}} & r_{s_{k-1}} \end{bmatrix}^T$$

$$x_{s_{k-1}} = \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T$$

Model input

$$u_{s_k} = \nu_d = \begin{bmatrix} u_d & v_d & w_d & r_d \end{bmatrix}^T$$

Model noise

$$w_{s_k} = \dot{\nu}_{s_k}$$

Motion model

$$x_{s_k} = f_s(x_{s_{k-1}}, u_{s_k}, w_{s_k}) = \begin{bmatrix} \eta_{s_{k-1}} \oplus (\nu_{s_{k-1}} \Delta t + \frac{1}{2} w_{s_k} \Delta t^2) \\ \nu_{s_{k-1}} + K(\nu_d - \nu_{s_{k-1}}) + w_{s_k} \Delta t \end{bmatrix}$$

$$K = diag(k_1, k_2, k_3, k_4) \quad k_i > 0$$

Dead Reckoning

4 DOF AUV Simulation

Algorithm 1: AUV4DOFSimulatedRobot Class

Class *AUV4DOFSimulatedRobot(SimulatedRobot)*

  Method `AUV4DOFSimulatedRobot(`$xs0, map$`)`

    $parent.Constructor(xs0, map)$; `// Call parent Constructor`

    $self.Qsk = Diag([\sigma_{\dot{u}}, \sigma_{\dot{v}}, \sigma_{\dot{w}}, \sigma_{\dot{r}}])$; `// Feature Map`

  end

  Method `fs(`$\boldsymbol{x_{s_{k-1}}}, \boldsymbol{u_{s_k}}$`)` $\rightarrow \boldsymbol{x_k}$

    $\eta_{s_{k-1}} = \boldsymbol{x_{s_{k-1}}}[0:4]$; `//` $x_{s_{k-1}} = \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T$

    $\nu_{s_{k-1}} = \boldsymbol{x_{s_{k-1}}}[4:8]$;

    $\nu_d = \boldsymbol{u_{s_k}}$; `//` $u_{s_k} = \begin{bmatrix} u_d & v_d & w_d & r_d \end{bmatrix}^T$

    $w_{s_k} = SampleGaussian(\mathbf{0}, self.Q_{s_k})$; `// Sample acceleration noise`

    $self.x_{s_k} = \begin{bmatrix} \eta_{s_{k-1}} \oplus (\nu_{s_{k-1}} \Delta t + \frac{1}{2} w_{s_k} \Delta t^2) \\ \nu_{s_{k-1}} + K(\nu_d - \nu_{s_{k-1}}) + w_{s_k} \Delta t \end{bmatrix}$;

    return $self.x_{s_k}$

  end

  Method `ReadCompass()`

    $w_c = SampleGaussian(0, \sigma_c)$;

    return $self.xsk[3] + w_g$

  end

  Method `ReadGyro()`

    $w_g = SampleGaussian(0, \sigma_g)$;

    return $self.xsk[7] + w_g$

  end

  Method `ReadDVL()`

    $w_g = SampleGaussian(\mathbf{0_{3\times 1}}, R_{dvl})$;

    return $self.xsk[4:7] + w_{dvl}$

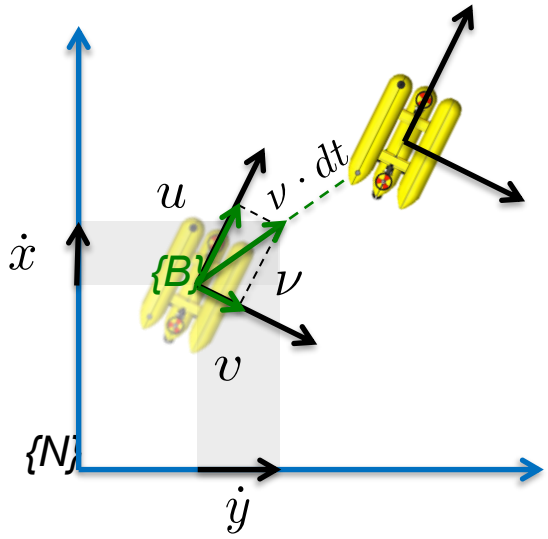
  end

Dead Reckoning

4 DOF AUV DeadReckoning

Algorithm 1: DR_4DOFAUV_DVLGyro Class

1 **Class** *DR_4DOFAUV_DVLGyro:Localization*
2 **Method** `Localization`$(index, kSteps, robot, x_0)$
3 $parent.Constructor(index, kSteps, robot, x_0)$;
4 **end**
5 **Method** `GetInput()` $\rightarrow \boldsymbol{u_k}$
6 $u_{dvl} = self.robot.ReadDVL()$;
7 $u_g = self.robot.ReadGyro()$;
8 $u_k = \begin{bmatrix} u_{dvl}^T & u_g \end{bmatrix}^T$;
9 **return** u_k
10 **end**
11 **Method** `Localize`$(\boldsymbol{x_{k-1}}, \boldsymbol{u_k}) \rightarrow \boldsymbol{x_k}$
12 $\eta_{k-1} = x_{k-1}$;
13 $\nu_k = u_k$;
14 $\eta_k = \eta_{k-1}.oplus(\nu_k \cdot dt)$; // $\eta_k = \eta_{k-1} \oplus \nu_k \cdot dt$
15 **return** η_k
16 **end**



Dead Reckoning



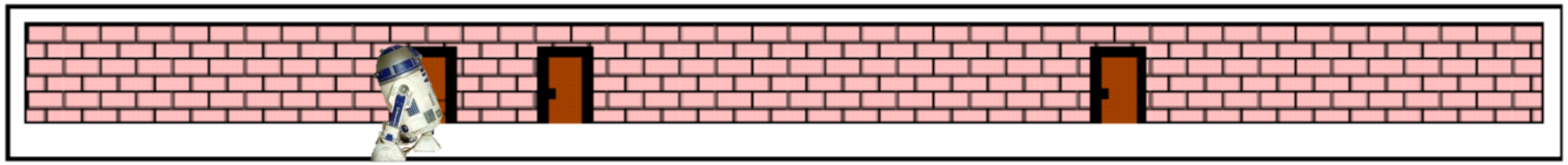
3 DOF Mobile Robot



4 DOF AUV

Will be implemented in the lab

Example I: Mobile Robot Localization in a Hallway



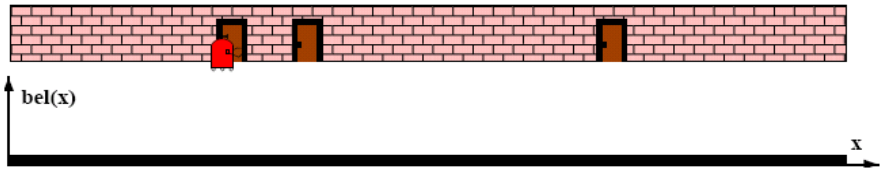
- One dimensional hallway
- Indistinguishable doors
- Position of doors is known (Map)
- Initial position unknown
- Initial heading is known

- **Goal**: Find out where the robot is

Example: Robot Localization With a Priori Map

- **Markov Localization**

Same probability of being in any x



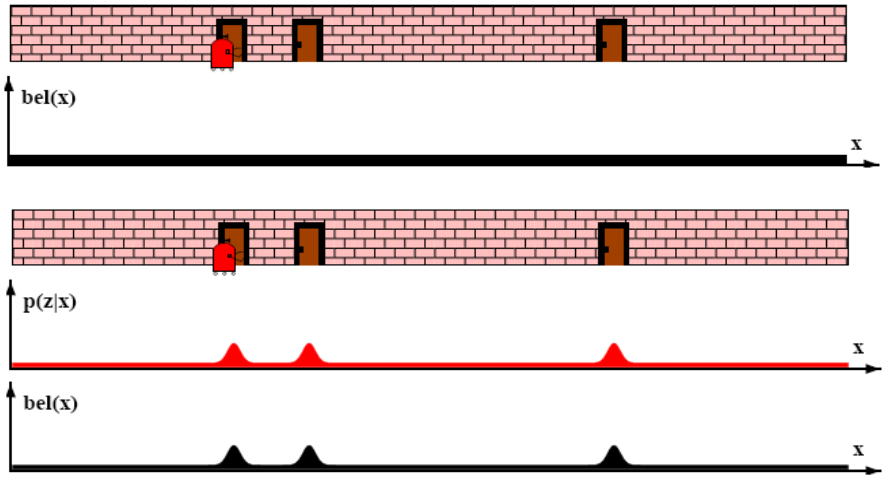
Example: Robot Localization With a Priori Map

- **Markov Localization**

Same probability of being in any x

The Robot senses a door

The belief over the position is updated



Example: Robot Localization With a Priori Map

- **Markov Localization**

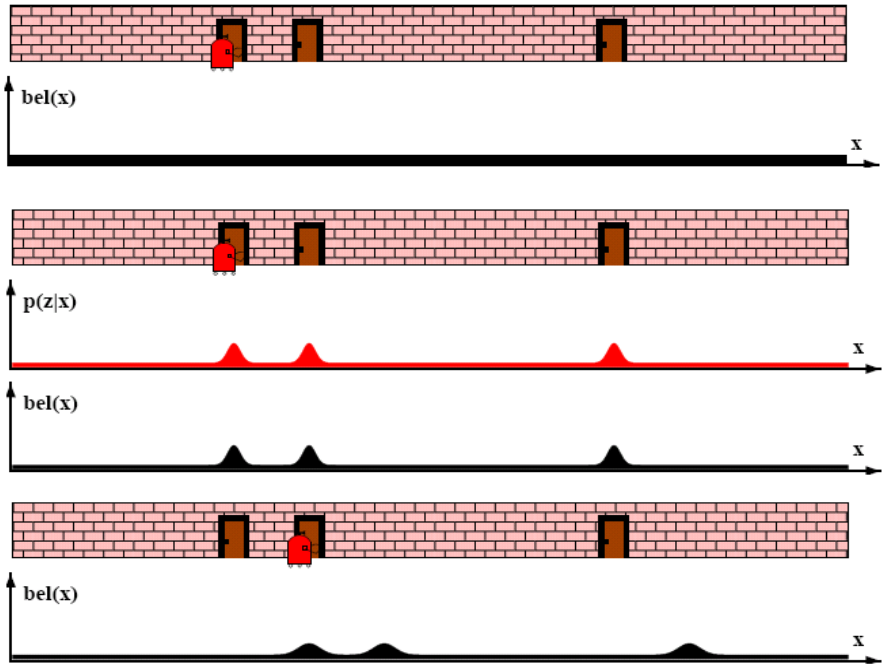
Same probability of being in any x

The Robot senses a door

The belief over the position is updated

The Robot Moves

The belief over the position is updated



Example: Robot Localization With a Priori Map

- **Markov Localization**

Same probability of being in any x

The Robot senses a door

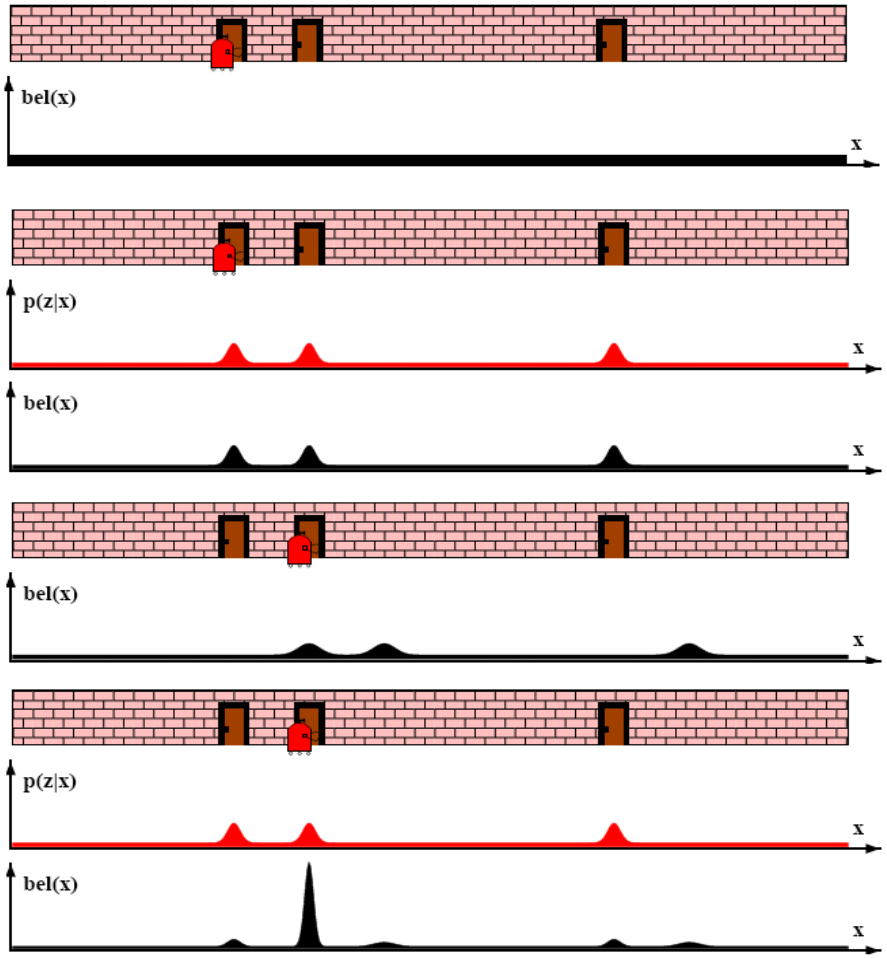
The belief over the position is updated

The Robot Moves

The belief over the position is updated

The Robot senses a door

The belief over the position is updated



Example: Robot Localization With a Priori Map

- **Markov Localization**

Same probability of being in any x

The Robot senses a door

The belief over the position is updated

The Robot Moves

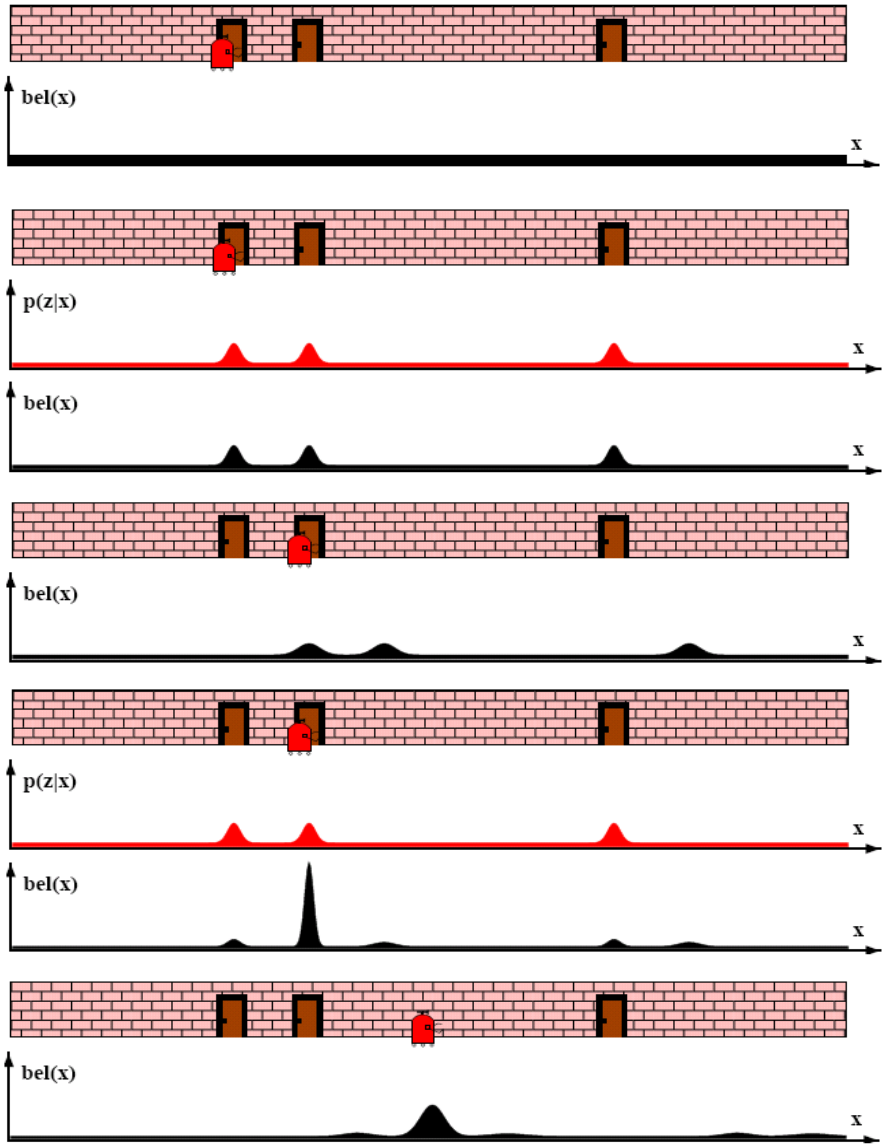
The belief over the position is updated

The Robot senses a door

The belief over the position is updated

The Robot Moves

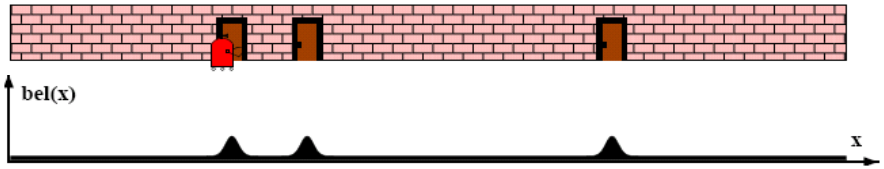
The belief over the position is updated



Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$



Introduction

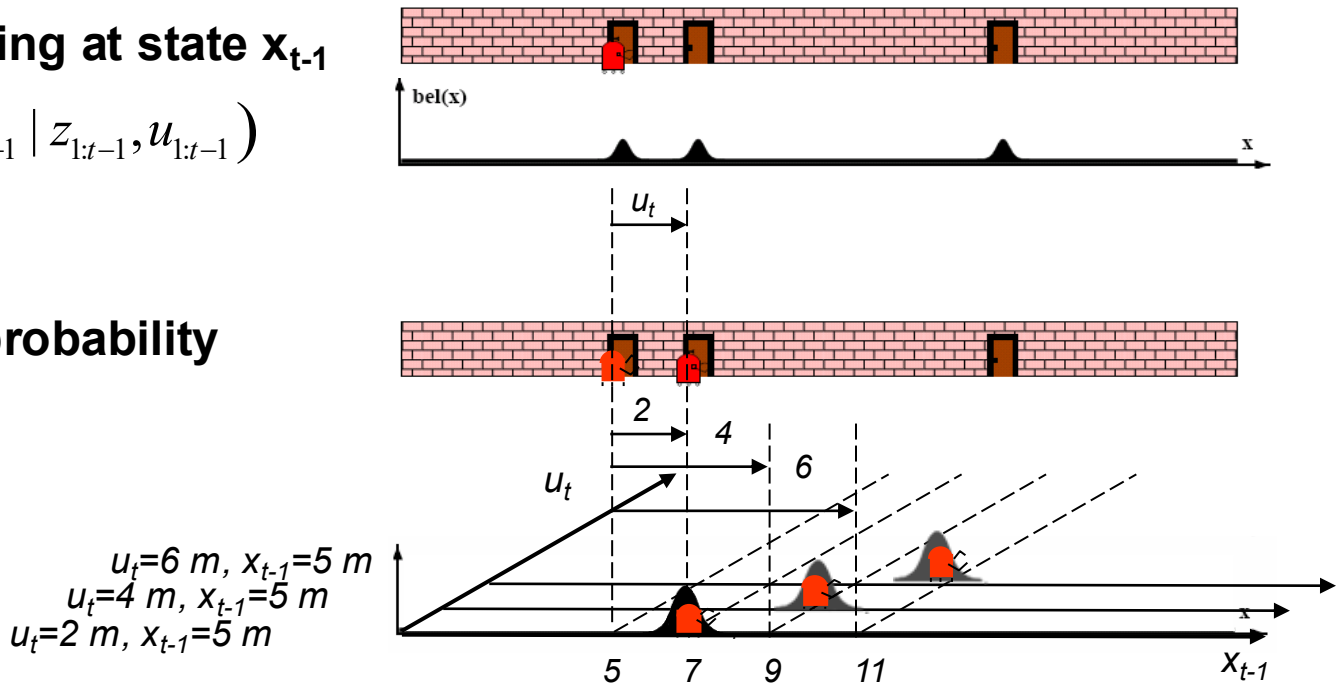
Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$

State Transition probability

$$p(x_t \mid u_t, x_{t-1})$$



Introduction

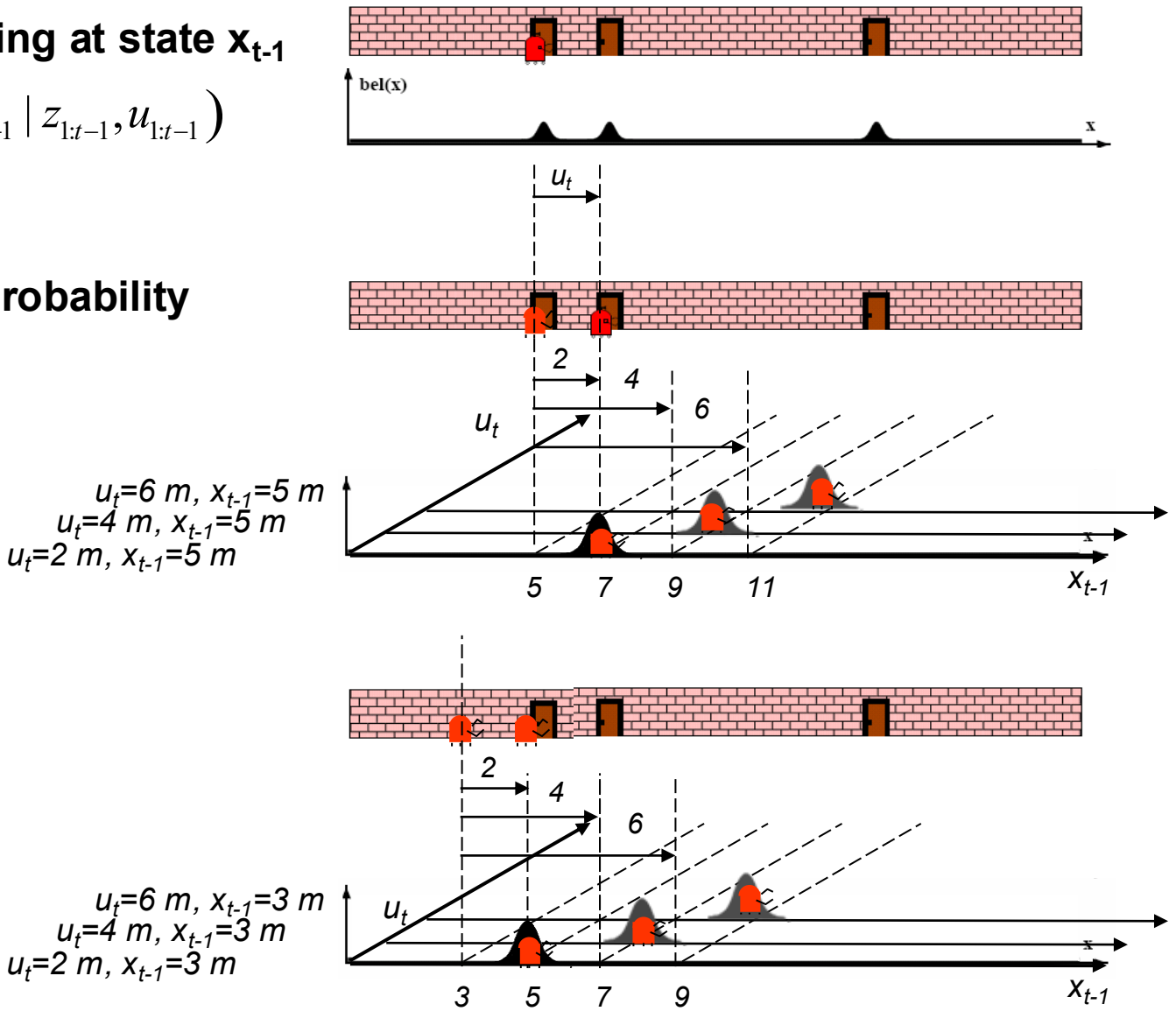
Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$

State Transition probability

$$p(x_t \mid u_t, x_{t-1})$$

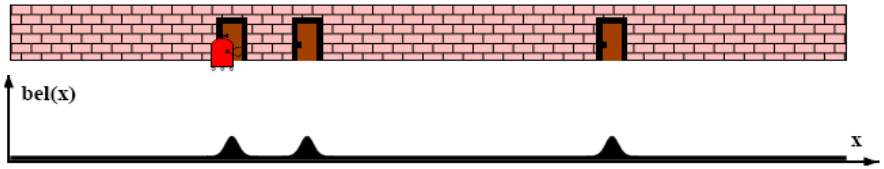


Introduction

Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

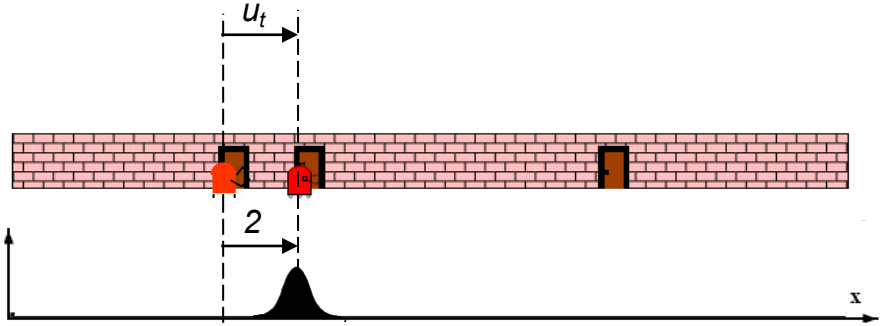
$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$



State Transition probability

$$p(x_t \mid u_t, x_{t-1})$$

u_t=2 m, x_{t-1}=5 m

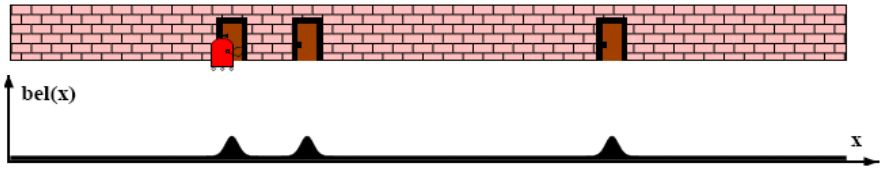


Introduction

Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

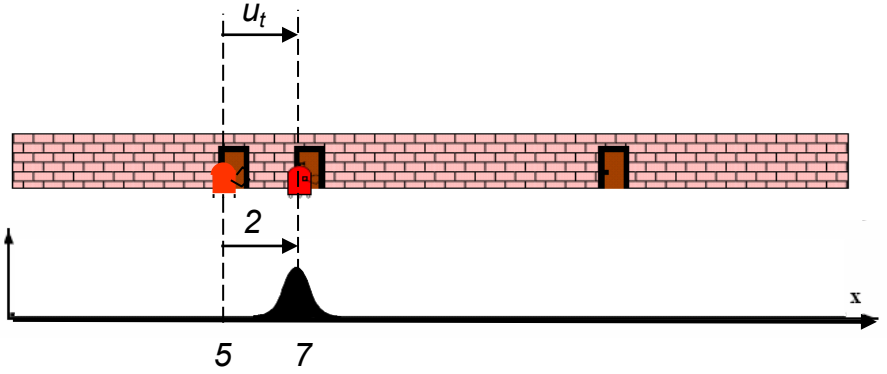
$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$



State Transition probability

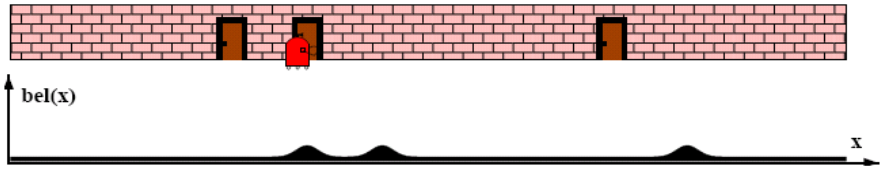
$$p(x_t \mid u_t, x_{t-1})$$

u_t=2 m, x_{t-1}=5 m



Prior Belief. Prediction of state x_t.

$$\overline{bel}(x_t) \quad = \quad p(x_t \mid z_{1:t-1}, u_{1:t})$$

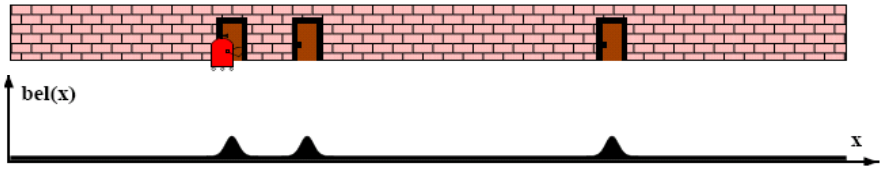


Introduction

Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

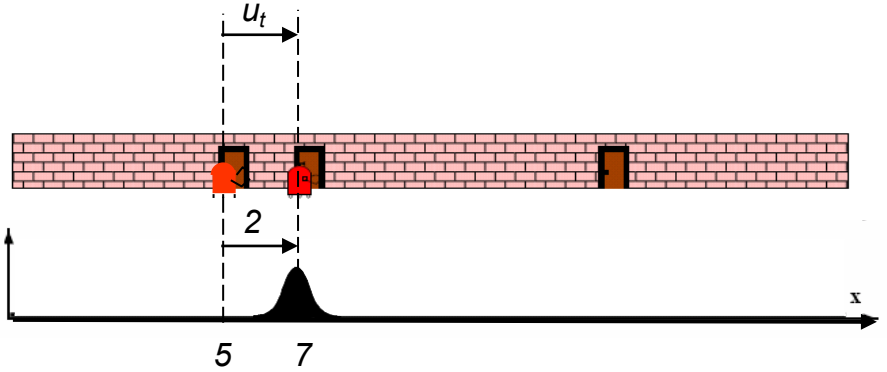
$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$



State Transition probability

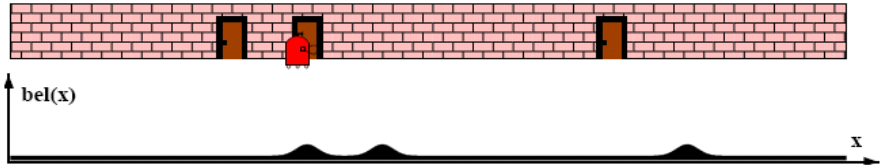
$$p(x_t \mid u_t, x_{t-1})$$

u_t=2 m, x_{t-1}=5 m



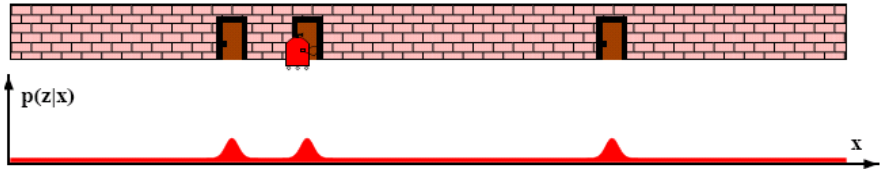
Prior Belief. Prediction of state x_t.

$$\overline{bel}(x_t) \quad = \quad p(x_t \mid z_{1:t-1}, u_{1:t})$$



Measurement probability

$$p(z_t \mid x_t)$$



Robot belief of being at state x_t

$$bel(x_t) \quad = \quad p(x_t \mid z_{1:t}, u_{1:t}) \; .$$

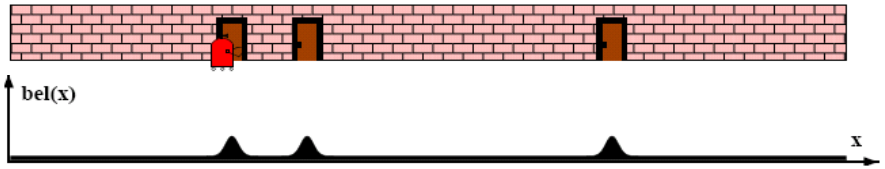


Introduction

Example: Robot Localization With a Priori Map

Robot belief of being at state x_{t-1}

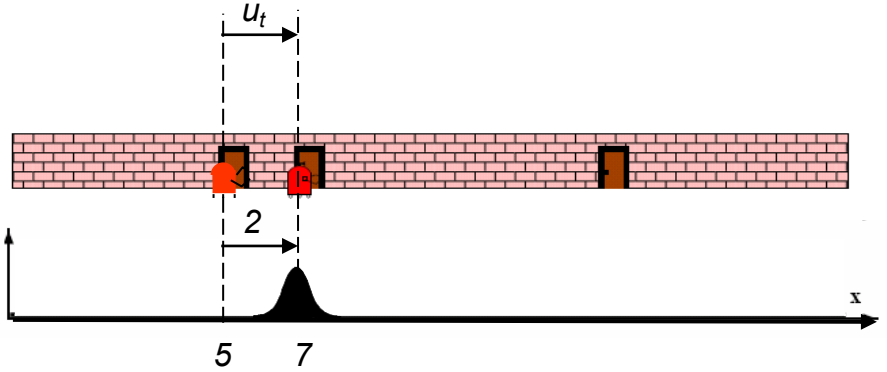
$$bel(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$



State Transition probability

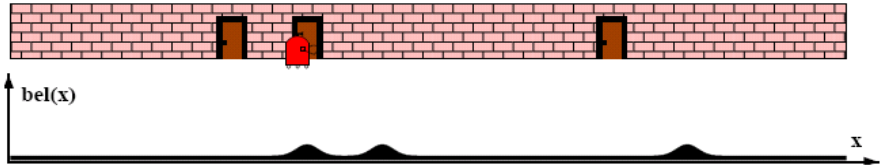
$$p(x_t \mid u_t, x_{t-1})$$

u_t=2 m, x_{t-1}=5 m



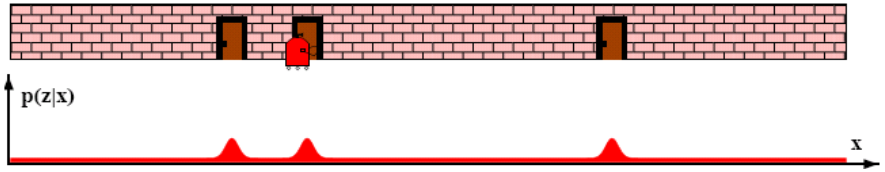
Prior Belief. Prediction of state x_t.

$$\overline{bel}(x_t) \quad = \quad p(x_t \mid z_{1:t-1}, u_{1:t})$$



Measurement probability

$$p(z_t \mid x_t)$$



Robot belief of being at state x_t

$$bel(x_t) \quad = \quad p(x_t \mid z_{1:t}, u_{1:t}) \; .$$



Introduction

- **Example**
  - Odometry motion model
  - Unknown initial position
  - Undistinguisable doors
  - Grid Localization: pdf implemented using histograms

